Implementation of a Privacy-Preserving Machine Learning Model Using Homomorphic Encryption

Thomas Green

***Abstract*—Modern machine learning systems pose significant privacy risks as they require access to raw user data during training and inference, exposing personally identifiable information, health records, and intellectual property to potential breaches and regulatory violations. This project investigates the feasibility of privacy-preserving machine learning using Fully Homomorphic Encryption (FHE), a cryptographic technique enabling computations directly on encrypted data without decryption. In business, this would provide measurable economic benefits by allowing lines of business and third parties to perform big data analytics on encrypted data while maintaining privacy and compliance records. The research implements an end-to-end system demonstrating encrypted inference across three distinct ML pipelines: MNIST handwritten digit classification, face detection with bounding box localization, and image border augmentation. The system employs dual-server architecture separating trusted and untrusted components. The untrusted server performs inference on encrypted data using TenSEAL wrapped neural networks, never accessing plaintext inputs, while the trusted server handles encryption and decryption operations with security key management. Each pipeline was implemented in both plaintext and encrypted variants using PyTorch and TensorFlow, enabling direct performance comparison. A web-based interface demonstrates real-world applicability, allowing users to interact with privacy-preserving ML models through intuitive tools. Evaluation focused on accuracy, performance trade-offs, and practical viability. Results confirm that encrypted inference maintains prediction accuracy equivalent to plaintext models while introducing significant computational overhead. The project explores FHE parameter tuning (polynomial modulus degree, coefficient modulus bit sizes) to optimize the security-performance balance. This work provides a proof-of-concept demonstration that FHE-based ML-as-a-service is technically feasible under current constraints, while highlighting performance limitations that must be addressed for broader production deployment. Future directions include encrypted training, federated learning integration, and exploration of hybrid privacy-preserving approaches.**

***Index Terms*—Fully Homomorphic Encryption, TenSEAL, CKKS scheme, Machine Learning, Encrypted inference**

I. Introduction

As of 2025, the increasing integration of machine learning into critical sectors has brought significant advancements. The capabilities and data-driven insights that ML provides for businesses are incredibly attractive but have also raised concerns regarding data security and user privacy. This has become particularly true in fields such as healthcare, finance, and cloud services, where sensitive information is routinely processed using AI models hosted on external or untrusted infrastructure. This challenge is not merely technical but existential for many ML applications today and raises the question of how organizations can leverage the potential of machine learning while maintaining the confidentiality, integrity and privacy of the data that drives it. Traditional machine learning workflows operate on a premise that is untenable, which is that data must be exposed or in real format for computation to be effective. This architectural decision creates an inherent vulnerability window where sensitive data or raw inputs must often be decrypted before processing, creating a critical vulnerability window where data is exposed. Beyond the risk of external attackers, there are also privacy concerns regarding the AI service providers themselves. Providers may log, repurpose, or monetize client data without explicit consent, creating long-term risks of data leakage, surveillance, or resale. This highlights that even when infrastructure is secure from intrusions, data confidentiality may still be compromised at the provider level. In distributed or cloud-based ML systems, this exposure is amplified as data traverses' multiple environments, crosses organizational boundaries, and persists in memory or storage in third-party infrastructure. The result is a growing attack surface where data breaches, insider threats, misconfigurations, and unauthorized access can compromise sensitive information at various points in the ML pipeline. Another challenge is the computational burden of strong cryptographic protections such as FHE. While theoretically appealing, FHE has historically been dismissed as impractical due to orders-of-magnitude slowdowns compared to plaintext ML inference and large ciphertext sizes that strain memory and computing resources. Recent advances in libraries and hardware accelerators, however, are beginning to close this feasibility gap, motivating a re-examination of FHE in applied machine learning.

ML has evolved from experimental technology to a mission-critical component of modern infrastructure and is becoming increasingly integrated into business operations and decision-making. The scale and sensitivity of data flowing through ML systems have grown exponentially, and with it, the consequences of failure. Recent incidents underscore the severity of these risks. In January 2025, a misconfigured cloud database at AI firm DeepSeek left over a million sensitive entries, including chat logs, API keys, and internal metadata, publicly accessible, raising major concerns about regulatory compliance and data security [1]. This incident is not isolated as it represents a pattern of vulnerabilities inherent to systems that rely on plaintext data processing. According to IBM's 2024 Data Breach Report [2], the global average cost of a single data breach reached USD 4.88 million, with financial industry enterprises spending USD 6.08 million per incident. Such incidents are particularly problematic in contexts where legal, compliance, and ethical standards require the safeguarding of personally identifiable information (PII), protected health information (PHI), intellectual property (IP), or confidential

corporate data.

Beyond the identified security concerns, the current modern machine learning workflows have created a broader crisis of trust. These workflows face a growing tension between data-driven innovation and the privacy rights of individuals and organizations whose data fuels these systems. Whether deployed as public-facing AI products or internal enterprise tools, effective ML models require access to large volumes of data, much of which may include personal, sensitive, or proprietary information. However, how this data is collected, shared, or processed frequently raises legal, ethical, and operational challenges.

Many large-scale models are trained on datasets scraped or aggregated from public and private online and internal sources, often without clear permission or informed consent from data owners [3]. This practice has led to class action lawsuits, regulatory scrutiny, and growing concerns over data ownership and transparency [4]. Individuals and organizations typically have no visibility or control over how their data is appropriated for AI development, resulting in a "trust deficit" in AI systems. This deficit is driven by legitimate fears that sensitive data may be misused, mishandled, or exploited without transparency or accountability. The legal and regulatory environment has evolved rapidly in response to these concerns with requirements that organizations must now navigate. In sectors such as healthcare, finance, and cloud services, these concerns are heightened by strict regulatory frameworks like the General Data Protection Regulation (GDPR) and the Health Insurance Portability and Accountability Act (HIPAA) alongside industry specific compliance standards such as PCI DSS or payment data, SOC 2 for service organizations and emerging AI-specific regulations such as ISO/IEC 42001 and EU AI Act. As a result, there is an urgent need to re-evaluate how sensitive data is treated within AI pipelines and develop built-in privacy safeguards that align innovation with ethical and legal responsibilities. Organizations must reconsider how sensitive data is handled in ML workflows to maintain regulatory compliance, uphold ethical responsibilities, and reduce the risk of data misuse. There is a pressing need for built-in privacy safeguards that enable computation directly on protected data without requiring decryption, especially in high-risk ML workflows, while supporting meaningful model development and prediction.

Integrating Fully Homomorphic Encryption (FHE) into workflows offers a promising solution to these concerns. It enables arbitrary computation to be performed directly on encrypted data [5], eliminating the need to expose sensitive information during training or inference. Its capability aligns with zero-trust principles and modern privacy requirements, especially in AI workflows that operate across distributed or third-party environments. Since its conceptual inception in 1978 [6] and practical breakthrough in 2009 [5], FHE has faced questions about its practicality and feasibility due to high computational overhead. This project investigates whether FHE can be practically and effectively applied to a standard ML workflow, balancing theoretical security with real-world usability. A prototype will be developed using open-source FHE libraries and public datasets to evaluate performance, accuracy, and resource use in end-to-end encrypted machine learning.

A. Goals

The aim of this project is to develop an end-to-end machine learning pipeline using Fully Homomorphic Encryption. The overall goal is to assess the capabilities of current libraries and implementations as well as the feasibility and practicality of FHE. This was performed across four development cycles, contributing to the outcome that highlights the feasibility and future potential of deploying FHE in real-world AI systems. The main contributions of this project are; (1) evaluation on current FHE libraries, assessing their capabilities and opportunities, construction of a ML pipeline that can support end-to-end workflow, (2) construction of three purpose-built FHE+ML models, that follow classification and regression supervised learning techniques, (3) analysis of models is performed on evaluation metrics as well as analysis between plaintext and encrypted outputs, (4) perform application development to showcase and demonstrate how encrypted inputs are processed, and predictions are returned securely, (5) evaluate model based on baselines established, perform model tuning (epochs, loss functions, learning rates) and evaluate established FHE metrics.

B. Requirements

The identified functional requirements (FR) and non-functional requirements (NFR) for the solution and deliverables are:

- **FR1**: A trained ML model that accepts encrypted input data and produces encrypted predictions. Training will be conducted on plaintext and encrypted data where appropriate, while inference remains fully encrypted.
- **FR2**: Implementation of encrypted arithmetic operations, i.e., addition, multiplication, using either open-source FHE libraries or a valid custom implementation that demonstrates correct fully homomorphic behaviour.
- **FR3**: Use of Scikit-learn for initial ML model development; optional use of Torch or TensorFlow for experimenting with deeper or quantized models.
- **FR4**: Benchmarking of plaintext versus. encrypted models based on metrics, such as; execution time, accuracy, memory usage, and encryption overhead.
- **FR5**: A simple user interface to demonstrate how encrypted inputs are processed, and predictions are returned securely. Intended for demonstration to an audience.
- **FR6**: Exploration of optimizations such as batching, approximate functions, federated training, or reduced circuit depth to improve feasibility and efficiency.
- **NFR1**: End-to-end encryption of inputs, model parameters, and outputs using lattice-based schemes. Adherence to zero-trust principles.
- **NFR2**: Ensure the system design aligns with data protection standards, such as GDPR.
- **NFR3**: Output must be interpretable and actionable by the user. If a GUI is implemented, it should prioritize clarity and simplicity.

- **NFR4**: The prototype should be extensible to support more complex models or larger datasets in future iterations.
- **NFR5**: Code will be modular and documented to support future modifications or library upgrades.

C. Benchmarking

This project benchmarks plaintext model pipelines against their FHE-encrypted counterparts to evaluate the feasibility and performance trade-offs of privacy-preserving machine learning. Comparison metrics include inference accuracy, computational overhead, end-to-end latency, and resource consumption. For classification tasks (MNIST), benchmarking compares accuracy, precision, recall, F1-score, and confusion matrix results between plaintext and encrypted inference. For regression tasks (face detection and border generation), benchmarks include Mean Squared Error, Mean Absolute Error, R^2 score, and inference quality metrics comparing plaintext and FHE implementations. Computational performance is benchmarked through inference time measurements, system resource utilization (e.g. CPU, memory), and FHE-specific encryption scheme configuration parameters. The project emphasizes inference, rather than solely optimizing inference performance.

II. Background Research

Understanding the intersection of privacy, cryptography, and machine learning is essential to evaluating the feasibility and impact of this project. This section synthesizes key academic and industry research on Fully Homomorphic Encryption (FHE), its applications in secure computation, and current challenges in privacy-preserving machine learning.

A. Literature Review

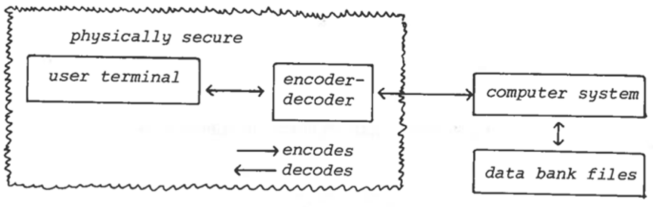

Fig. 1. Initial Homomorphic Design [6]

One of the earliest conceptual breakthroughs in privacy-preserving computation was proposed by Rivest, Adleman, and Dertouzos in their 1978 report [6]. In this foundational work, they introduced the term "privacy homomorphism" to describe encryption functions that allow computations to be performed directly on ciphertext, producing encrypted results that, when decrypted, match the outcome of operations as if they had been performed on the plaintext. This idea emerged from a practical small application use case. The idea was a loan company that uses a commercial time-sharing service to store its records (Figure 1) [6]. The loan company's "data bank" contains sensitive information that should be kept private. On the other hand, suppose that the information protection techniques employed by the time-sharing service are not considered adequate by the loan company; specifically, the systems programmers would have access to the information. In response to this, the loan company decides they want to encrypt all their data kept in the "bank" and to maintain a policy of only decrypting data at the home office, meaning the time-shared computer will never decrypt data. To demonstrate the feasibility of encrypted computation, the authors designed a set of example encryption functions, termed privacy homomorphisms, that allowed basic arithmetic operations to be performed directly on the ciphertext. These were implemented using algebraic systems that preserved operational consistency between plaintext and encrypted data. The authors formalized the mathematical structure of homomorphic encryption using algebraic systems and outlined its potential for secure remote computation while acknowledging inherent limitations, particularly in supporting secure comparisons and defending against chosen-plaintext attacks. Although the specific schemes proposed were later found to be insecure [6], their work laid the theoretical groundwork for decades of subsequent research, including the development of modern FHE.

The concept of a Fully Homomorphic Encryption scheme was introduced by Craig Gentry in his 2009 PhD thesis [5], which presented the first construction of an FHE scheme that was provably secure and theoretically sound. An FHE scheme enables arbitrary computations on encrypted data, supporting both addition and multiplication. Denoted as, given messages m_1 and m_2 and an encryption function Enc, and computationally feasible functions f and f' for ciphertext and plaintext respectively, homomorphic encryption satisfies $f(\text{Enc}(m_1), \text{Enc}(m_2)) = \text{Enc}(f'(m_1, m_2))$. What makes such a scheme fully homomorphic is the ability to evaluate its own decryption function, achieved through a technique known as bootstrapping. Gentry's model began with developing a somewhat homomorphic "bootstrappable" encryption (SHE) scheme that could perform a bounded number of additions and multiplications on ciphertexts. The innovation came in Gentry's technique of bootstrapping, a recursive process wherein the scheme homomorphically evaluates its decryption circuit to "refresh" ciphertexts and reduce accumulated noise [5]. This allowed the transformation of a SHE into a fully homomorphic one capable of evaluating arbitrary circuits. The construction was lattice-based, relying on ideal lattices and the hardness of problems such as the Shortest Vector Problem (SVP) [7] and the Ideal Coset Problem, making the scheme resistant to both classical and quantum attacks. Gentry's blueprint also emphasized the importance of keeping the decryption circuit within the complexity class NC^1 [5], ensuring it could be evaluated homomorphically. Nicks Class or NC^1, is a class of problems solvable by circuits of logarithmic depth and polynomial size, making them efficiently parallelizable and feasible to evaluate homomorphically. While the original construction was computationally impractical due to the overhead introduced by bootstrapping and the depth of the evaluated circuits, it established the foundational paradigm, SHE + bootstrapping = FHE, which modern FHE research continues to build upon. In this context, a circuit refers to a sequence of arithmetic operations (e.g., additions, multiplications) performed on the encrypted

data, while the depth of a circuit is the maximum number of sequential multiplications. Each arithmetic operation increases the noise within a ciphertext, which is random data added during encryption to preserve security. If the noise grows too large, it prevents correct decryption. Gentry's original scheme required evaluating decryption circuits of depth $O(\lambda^5)$ and performing computations with encrypted secret keys of size $O(\lambda^7)$ leading to an overall complexity of quasi-linear in λ^9 per ciphertext refresh [5]. Here, λ denotes the security parameter, which governs the size of keys and ciphertext to achieve a target level of cryptographic strength. Gentry's work marked a transition from theoretical to practical exploration and remains the cornerstone of all subsequent lattice-based FHE developments. At the core of Gentry's proposal was the notion of evaluating a decryption circuit homomorphically on the ciphertext, thus resetting the noise, and enabling further computation. While theoretically well-designed, this process was slow and complex, requiring deep circuit evaluations and large lattice dimensions.

A key advancement in the path toward practical homomorphic encryption came with Brakerski, Gentry and Vaikuntanathan's 2011 work [8] on leveled Fully Homomorphic Encryption without bootstrapping, most known as leveled FHE or the BGV scheme. This scheme removed the need for expensive bootstrapping operations within bounded-depth computations. Instead of supporting arbitrary-length circuits like earlier FHE schemes, leveled FHE schemes are parameterized by a predefined depth L (number of sequential addition/multiplication operations), allowing encrypted computations on circuits up to L levels without refresh steps [8]. This significantly improves performance and opens new avenues for real-world implementation. Their scheme is built on the Learning with Errors (LWE) and Ring-LWE [9] assumptions, utilizing modulus switching and relinearisation to manage noise growth without relying on bootstrapping [8]. The technique replaces fully general circuit evaluation with a more practical and bounded-depth model, making it computationally feasible to run meaningful machine learning inferences on encrypted data. Limiting extensive bootstrapping also results in better latency and throughput in settings where the circuit depth is known in advance, a common case for many ML applications. This leveled approach laid the groundwork for most modern FHE libraries and influenced a wide range of practical applications by making FHE far more efficient and reproducible in constrained environments. A privacy-preserving ML model using leveled FHE is possible if the ML model's computations don't exceed the bounded-depth computations of the FHE model.

While leveled FHE represented a major practical advancement by eliminating bootstrapping for bounded-depth circuits, a fundamental limitation remained for real-world applications requiring approximate arithmetic. BGV and other early schemes were designed for exact computation over finite fields or integers, with decryption schemes like BFV (Brakerski-Fan-Vercauteran), and FV (Fan-Vercauteran) that place messages and noise in separate regions of the modulus space. This design ensured exact recovery of plaintexts but created severe efficiency problems for approximate computation. Since noise could corrupt the most significant bits of computational results, maintaining correctness required exponentially large ciphertext module as circuit depth increased. For a circuit of multiplicative depth L, the ciphertext modulus needs to grow as $O(t^L)$ to prevent noise from overwhelming the message space, where t is the plaintext modulus. This exponential growth made even moderately deep circuits computationally infeasible, limiting applications in machine learning, where approximate results are acceptable and exact arithmetic is unnecessary. The CKKS scheme was introduced in a 2017 paper [10] by Cheon, Kim, Kim, and Song, where rather than treating encryption noise as an obstacle to be contained, CKKS embraces it as part of the inherent error in approximate computation. The scheme features a simplified decryption structure, $\langle C, SK \rangle = m + e \pmod q$, where the noise is small relative to the message m [10]. This noise, originally added for security based on the Ring-LWE hardness assumption, is treated as equivalent to rounding errors in standard floating-point arithmetic. When e is sufficiently small compared to the significant figures of m (meaningful digits of precision in the message), the decrypted value $m' = m+e$ serves as an acceptable approximation of the original message[10]. The innovation enabling practical CKKS implementation is the rescaling procedure. In unencrypted approximate arithmetic, maintaining manageable number sizes requires periodic rounding to discard least significant bits. CKKS implements an operation that rounds the encrypted message by removing its least significant bits while reducing the ciphertext modulus, where instead of managing the noise to maintain security, it manages message magnitude to enable continued computation[10]. A critical consideration in CKKS is the management of computational depth and the remaining levels. The depth of a computation, fixed at parameter selection, determines the maximum number of multiplicative operations that can be performed before noise renders decryption unreliable. The level indicates how many multiplication operations remain available and decreases dynamically as multiplications are executed. Each multiplication consumes one level, and rescaling is typically performed immediately afterward to manage both message magnitude and noise growth. If computations exceed the available depth of levels, excessive noise accumulation prevents correct decryption.

The implementation of "Designing more versatile homomorphic encryption-based CNNs" was performed by Choi et al. in 2024 [11]. In their work, the authors addressed the limitations of existing homomorphic encryption based convolutional neural networks (CNNs) by designing a framework, UniHENN, which can support more complex and diverse CNN architectures under encryption. They began by contrasting two of the most widely used homomorphic encryption (HE) schemes, BGV and CKKS, both of which have distinct computational properties and data representations. BGV supports integer-based arithmetic with exact results, making it suitable for discrete computations but restrictive for deep learning tasks involving floating-point or real-valued inputs [11]. In contrast, CKKS supports approximate arithmetic on real and complex vectors, which aligns better with the numerical operations used in CNNs [11]. This makes CKKS more attractive for encrypted deep learning despite trade-off of slight numerical approximation errors. In their implementation, Choi et al. se-

lected the CKKS scheme and highlighted its three fundamental encrypted operations: addition, multiplication, and rotation of ciphertexts. To maximize performance, they emphasized the need to carefully design a plaintext structure that mirrors the input data structure ensuring encrypted inputs maintain the same layout as the original tensors, enabling efficient batching and layer-wise transformations during CNN inference [11]. In CKKS, the number of slots, which determines how many real numbers can be packed into a single ciphertext, is defined during parameter selection. Letting N denote the total degree of the CKKS polynomial modulus, the number of slots available for packing equals $N/2$. This structural property enables vectorized operations, where entire arrays of values can be encrypted and manipulated simultaneously. Choi et al. formalized the supported operations as follows. For two real vectors V_1 and V_2, their encrypted forms are denoted as $C(V_1)$ and $C(V_2)$ [11]. The notation (P) refers to an operation between a ciphertext and a plaintext, while (C) indicates an operation between two ciphertexts. For addition with plaintext: $\text{Add}(C(V_1), V_2) = C(V_1 + V_2)$, addition with ciphertext: $\text{Add}(C(V_1), C(V_2)) = C(V_1 + V_2)$, multiplication with plaintext: $\text{Mul}(C(V_1), V_2) = C(V_1 \cdot V_2)$, multiplication with ciphertext: $\text{Mul}(C(V_1), C(V_2)) = C(V_1 \cdot V_2)$ [11]. For rotation, where $v = (U_1, U_2, \ldots, U_{N/2})$ represents the plaintext vector packed into a ciphertext, the rotation operation by a positive integer r can be expressed as: $\text{Rot}(C(v), r) = C(U_r, U_{r+1}, \ldots, U_{N/2-1}, U_0, \ldots, U_{r-1})$ [11]. This effectively performs a cyclic permutation of the slots, which is essential for implementing convolutions and matrix multiplications under encryption. The addition and multiplication in the plaintext domain are defined elementwise, enabling SIMD (single instruction, multiple data) processing across all packed slots. A crucial consideration discussed by Choi et al. is the management of depth and level within the CKKS scheme. The depth refers to the maximum number of multiplicative operations that can be performed on a ciphertext before decryption becomes unreliable due to noise accumulation. This value is fixed at the time of parameter selection. The level indicates the number of remaining multiplicative steps that can be executed during computation. While the depth is static the level decreases dynamically as multiplications are carried out. Exceeding the allowed multiplicative depth or mismanaging the level can result in excessive noise rendering the ciphertext un-decryptable. Therefore, parameter tuning and careful network design are essential to maintain accuracy and performance in encrypted CNN inference. Choi et al. validated UniHENN's accuracy by comparing encrypted and plaintext inference results using two CNN models on MNIST and CIFAR-10, confirming no significant loss in performance [11]. They further examined how CKKS parameters affect inference finding that increased depth linearly raises computation time, while higher scale improves accuracy but reduces available depth [11]. This revealed a trade-off between precision and computational feasibility. UniHENN also does not use bootstrapping and GPU support, limiting its applicability to deeper models and real time scenarios. The authors suggest future enhancements such as bootstrapping integration and GPU acceleration to address these challenges.

While UniHENN demonstrated encrypted CNN inference with parameter management, practical deployment requiring addressing implementation challenges in real systems, Helbawy, Bahaa, and El Bolock's 2023 CryptonDL [12] work provided insights into building encrypted classification systems using TenSEAL, a Python wrapper of Microsoft SEAL that supports CKKS. The system employed a common two-phase workflow; training a conventional CNN on unencrypted data using PyTorch, then transferring learned weights to an encrypted CNN performing inference entirely on CKKS-encrypted inputs with only final predictions decrypted [12]. This approach addressed a fundamental FHE limitation that training requires access to ground truth labels and gradients for backpropagation which remains computationally infeasibly in encrypted form. They evaluated on three 28×28 grayscale datasets (MNIST, Fashion-MNIST, Kuzushiji-MNIST) where it achieved accuracies of 98.32%, 88%, and 92% respectively through architectural optimizations included increased convolutional channels (4-12) and adjusted kernel sizes [12]. The encrypted accuracy closely matched unencrypted performance demonstrating CKKS's approximate arithmetic introduced minimal degradation for these tasks. Results highlighted substantial computational overhead inherent to homomorphic encryption. While unencrypted inference completed in under 1 second, encrypted evaluation required 2-6 hours for 10,000 images [12]. Several limitations temper practical integration as evaluation used only shallow networks (one convolutional layer, two fully connected layers) on small grayscale images, whereas modern CNN applications employ deeper architectures that have multiple hidden layers and multiple convolutions, as well as high channel RGB data (1 channel greyscale, 3 channel RGB) demanding significant multiplicative depth [12]. Sequential processing with batch size one during encrypted inference foregoes batching optimizations and the claim of succeeding without pre-trained models represents a semantic distinction since weights still are derived from plaintext training. CryptonDL's contribution lies in demonstrating that accessible tools like TenSEAL enable implementation without deep cryptographic expertise and that hyperparameter tuning improves accuracy within FHE constraints.

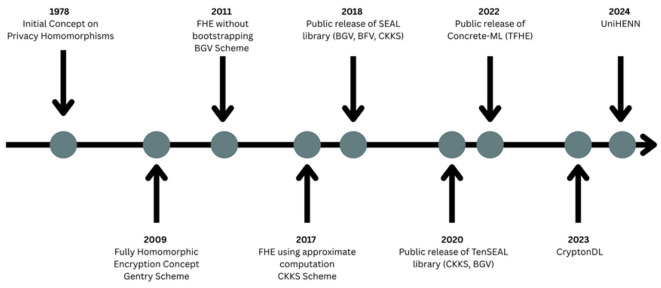

Fig. 2. Timeline of previous FHE papers and libraries

B. FHE Library Analysis

PySEAL is a Python wrapper around Microsoft's Simple Encrypted Arithmetic Library (SEAL) written in C/C++. It

TABLE I
COMPARISON OF FUNCTIONAL REQUIREMENTS ACROSS FHE LIBRARIES

Functional Requirements	PySEAL	TenSEAL	Concrete-ML
FR1	Y	Y	Y
FR2	Y	Y	Y
FR3	N	Y	Y
FR4	Y	Y	Y
FR5	N	N	N
FR6	Y	Y	Y

TABLE II
LEVELED AND BOOTSTRAP APPROACH CASES [18]

	BGV	CKKS	TFHE-LIB	TFHE-Concrete
Operations	Leveled	Leveled	Bootstrapped	Leveled + Bootstrapped
Non-Linear Functions	Approximate	Approximate	Exact	Exact or Approximate
Data Types	Integers	Reals	Boolean	Boolean + Integers

was introduced in March 2018 and is one of the most widely used homomorphic encryption frameworks. SEAL supports three major encryption schemes: BFV and BGV schemes, which allow modular arithmetic to be performed on encrypted integers [13] and CKKS scheme, which enables additions and multiplications on encrypted real or complex numbers [13]. These are somewhat homomorphic schemes that become fully homomorphic through bootstrapping. PySEAL and SEAL by default operate in a leveled FHE mode, meaning computations are limited to a fixed depth of circuit before decryption is required. This avoids the high computational cost that bootstrapping would provide in most practical scenarios. This makes them especially suitable for inference tasks where the depth of computation can be bound in advance. In PySEAL, encrypted values are stored as polynomials over rings, and computations directly manipulate these encrypted polynomials [14]. The system manages noise growth internally and the lack of bootstrapping means a carefully chosen encryption parameter set is essential to support the desired computation depth [14]. Although PySEAL exposes lower-level functionality compared to other libraries, it enables full customization of FHE workflows and is ideal for benchmarking FHE-based ML operations under controlled conditions, offering both clarity and reproducibility in a research setting.

TenSEAL, released in April 2021, is designed for machine learning applications that require encrypted inference while maintaining ease of integration with popular frameworks like PyTorch. TenSEAL uses the CKKS scheme and provides two primary encrypted tensor types: CKKSVector, which encrypts a vector of real values into a single ciphertext [15], and CKKSTensor, which supports N-dimensional real-valued tensors by organizing them into (N-1)-dimensional tensors of ciphertexts [15]. This enables efficient SIMD-style operations supporting operations over encrypted floating-point numbers. Reliance on CKKS makes it a leveled FHE approach [15], where users must set the multiplicative depth (or levels) during context creation, and once exhausted, no further operations can be performed without re-encryption or bootstrapping (which TenSEAL does not implement by default). This makes TenSEAL especially suitable for ML inference workflows where the number of operations is known and limited. Its support for encrypted tensors, vector-matrix multiplication, and dot products [16] enables private inference over real-valued vectors with minimal changes. However, the reliance on polynomial approximations for non-linear activations and manual noise management limits its use in deeper or more complex models [17]. Despite these constraints, TenSEAL enables practical encrypted inference in low-latency privacy scenarios, offering a compelling mix of cryptographic and engineering accessibility.

Concrete-ML, introduced in July 2022, is a library developed by Zama [18] that leverages TFHE (Fully Homomorphic Encryption + Torus, a structure for encoding real numbers modulo 1) to compile and run full machine learning workflows directly on encrypted data. Unlike leveled approaches, which rely on CKKS and BGV, TFHE supports programmable bootstrapping, which allows it to evaluate arbitrary functions, including non-linear ones, without switching to plaintext (Table 2) [18]. Concrete-ML implements a variant of TFHE that supports both leveled and fast bootstrapped operations, as well as approximate/exact evaluation of arbitrary functions [18], where computations are structured in a way to minimize depth while preserving correctness. It performs model quantization to convert real-valued models into integer-based equivalents, then compiles them into circuits executable under TFHE constraints [18]. This workflow includes both quantization-aware training and post-training quantization, depending on the model type, and culminates in FHE circuits that can be deployed in client-server settings [18]. Clients encrypt inputs locally, send them to a server for encrypted evaluation, and then decrypt the results, ensuring full data privacy throughout. While Concrete-ML imposes constraints on model complexity and precision, its transparent compilation stack and automated cryptographic handling make it one of the most production-ready FHE libraries available for practical ML deployment.

C. Machine Learning Techniques

There are several machine learning (ML) techniques that are designed for different use cases and data. Supervised

learning is where an ML model learns labelled training data, which is well known for specific tasks such as classification and regression. In supervised learning tasks, the target output is known during training y, where it is trying to find the connection from the input x.

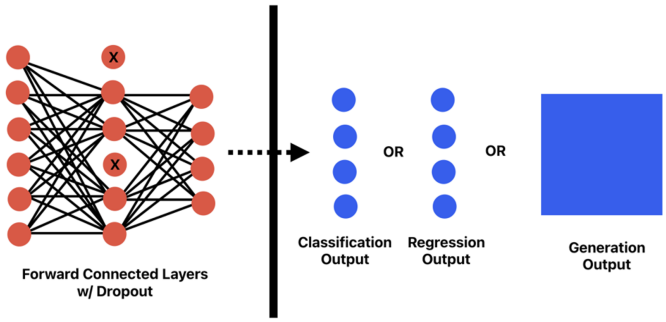

Fig. 3. Feedforward Connected Network mapping to Output(s)

A feedforward connected neural network (FCNN) (Figure 3) is a layered, fully connected architecture that defines a deterministic function f, to approximate an unknown target function, f^* [19]. In supervised learning we normally observe training data as paired samples (x_i, y_i) where each input $x_i \in \mathbb{R}^d$ [19] maps to an output $y_i = f^*(x_i)$ [19]. FCNN implements a parameterized function $f_\theta(x)$ [19] where θ denotes the learnable weights and biases. Each layer transforms its input according to $f^i(z^{(i-1)}) = g(W^i z^{(i-1)} + b^i)$ [19] where W^i is the weight matrix, $b^{(i)}$ the bias vector, and g a nonlinear activation function. The nonlinearities enable the network to capture complex, non-linear relationships in data. The parameters $\theta = \{W, b\}$ [19] are learned during training via backpropagation and gradient-based optimization to minimize a chosen loss function.

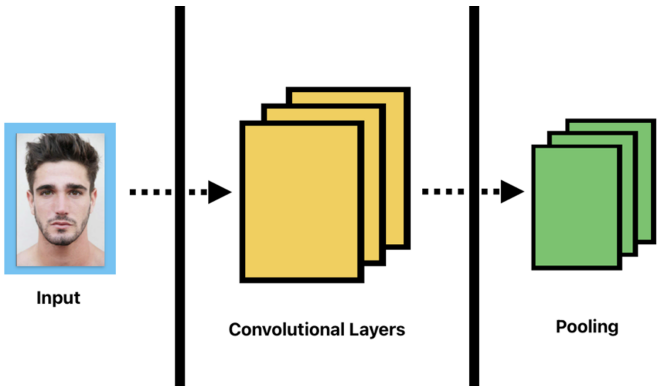

Fig. 4. Convolutional Neural Network (CNN) Structure

The algorithm chosen to use in this project is convolutional feedforward neural networks or CNN (Figure 4). Rather than relying on fully connected layers, we introduce convolutional layers that act over local receptive fields in the input. These layers perform local operations to capture spatial or sequential patterns, making them particularly effective for tasks involving images. Each convolutional layer learns a set of filters (kernels) that convolve across the input to detect low-level features such as edges, textures, or other spatial patterns. The result is then passed through nonlinear activation functions. While fully connected layers are still used in certain stages of the network, convolutional layers reduce the number of parameters by sharing weights across the input. This makes the model more efficient, especially when handling large-scale spatial data, and allows it to maintain spatial hierarchies. Discriminative models learn to directly predict an output y or class label given an input x, focusing on estimating $p(y|x)$ or decision boundaries. However, generative models instead aim to learn the underlying structure of the data distribution $p(x)$ or $p(x|z)$ and can create new statistical consistent samples.

Exploring different activation functions such as ReLU, Leaky ReLU, Square, ELU, GELU, etc., introduces non-linearity, enabling the network to model complex mappings. ReLU, $g(Wz) = \max(0, Wz)$ [20, is widely used for its simplicity and efficiency, but it can lead to inactive neurons. Leaky ReLU, $g(Wz) = \max(\alpha Wz, Wz), \alpha \ll 1$ [21], addresses this issue by allowing small gradients for negative inputs, improving gradient flow. Polynomial-friendly activation functions like square activation function, $f(x) = x^2$, also produces non-linearity enabling the network to learn complex relationships, however it has a linear derivative $(2x)$, which can lead to challenges in training deeper networks by causing unstable weight updates and lacks the saturation region of more complex non-linear functions.

While Adam will be used as the baseline optimizer for robust convergence and adaptive learning rates, other optimizers such as RMSprop, SGD and Adagrad will be explored to assess convergence and final model accuracy. Regression and Classification loss functions such as mean squared error, cross entropy, KLDivLoss will be explored to help normalize outputs. Mean squared error (MSE) loss,

$$L(\theta) = \frac{1}{|A|} \sum_{n=1}^{A} \|y^n - f(x^{(n)}; \theta)\|^2 \tag{1}$$

[22], measures the squared difference between predicted and actual outputs. Minimizing MSE not only leads to accurate predictions but is justified under the assumption of distributed errors as the maximum likelihood solution for targets. Cross entropy loss,

$$\text{loss} = -(T \cdot \log(Y) + (1 - T) \cdot \log(1 - Y)) \tag{2}$$

[23], where T is the true label and Y is the predicted probability, calculates the negative sum of the true probability of each class multiplied by the logarithm of its predicted probability, as well as enables quick training. If the model assigns a high probability to the correct class, cross entropy is low and vice versa.

Gradient based optimization will also be used, where parameter θ will be iterative updated in the direction that minimizes the loss function. Basic gradient descent updates weights using a fixed learning rate, while exploring optimization algorithms adapting learning rates dynamically to improve convergence and stability. Rather than computing the gradient over the entire dataset, training will use mini-batching, where gradients are computed over small, randomly sampled subsets of the full

dataset. This allows faster updates and efficient use of memory. The optimizer used adjusts the learning rates per parameter based on the first and second moments of the gradients. The general update rule can be represented as

$$\theta^{(k+1)} = \theta^k + \epsilon \nabla_\theta L(\theta \mid B^k) \tag{3}$$

[24], where ϵ is the learning rate, B^k is the kth mini-batch, and $\nabla_\theta L$ is the gradient of loss with respect to θ. During each iteration, the model computes the loss, using the loss function, only over that mini-batch, meaning that the epoch loss is not necessarily a true average over the entire dataset unless explicitly computed. Small batch sizes result in faster updates and introduce more noise in gradients, which may help with generalization. Larger batches offer more stable updates but require more memory and may converge slower.

III. Tools and Methodology

A. Methodology

The project adopted an Agile rapid prototyping methodology approach, organized into incremental development cycles. Each cycle includes initial planning, design, implementation, testing, benchmarking, and refinement based on observed results. This approach enabled quick and incremental experimentation with different combinations of ML models and FHE libraries, allowing the project to remain agile while evaluating the feasibility of encrypted inference in real-world scenarios. Several sprints were outlined at the beginning of the project that were built over 2–4-week periods ensuring maintained flexibility, speed and continuous learning were prioritized.

B. Tools

Project management is conducted through **GitLab**, where features are broken into granular issues linked to specific branches and merge requests. Each feature is developed independently and undergoes self-review before merging into the main branch to ensure clarity and accountability. Total of 186 commits from 24/03/25 to 19/09/25. Total of 23 issues created for tracking. Total of 11 branches/merge requests created for development across all 5 areas of the project. Allows for easy tracking and artifact management. The documentation directory contains artifacts that include agreements signed, designs made, health and safety documentation, images used/created for repository and markdown documents and user testing documentation. The meeting notes directory contains markdown files to keep track of notes taken during discussion with supervisor and helps iterate feedback provided every week. The node modules directory contains files required for web application management and production. The source directory contains the main components of the artifact, including plaintext and encrypted pipelines, library research performed on FHE libraries, initial testing/evaluation scripts, and web application code files for client.

Visual Studio Code (v1.105.0) was the primary development environment (IDE) used throughout the project. It's plugin and strong support ecosystem for Python development made it well-suited for implementing. Features such as integrated Git version control, debugging tools, and Python-specific extensions (e.g., Jupyter, Pylance, Python Debugger) enabled a streamline development workflow, enabling the iteration and rapid prototyping we were aiming for.

Initial development/research performed on FHE libraries Experiments were made using **Jupyter Notebooks** (v24.11.3). These notebooks act as documents that combine python code, outputs, and commentary, supporting transparency, reproducibility, and iterative evaluation. This helped benchmark and evaluated encrypted inference pipelines that will be later used during the implemented ML+FHE pipeline. This tooling supports the structured exploration of trade-offs between accuracy, performance, and security in different FHE-ML configurations. This will then be converted into a full Python program, serving as the final model and submitted prototype.

Other tools used in the development lifecycle include **Apple Freeform** (v4.0) which is a whiteboard tool on MacOS which was used for planning and design, including CRC and UML diagrams for pipelines and web application wireframes, and **Google Forms** which contained system usability scale questions for user testing volunteers.

C. Languages

Focus on evaluating languages was based on scalability, usability, and familiarity. For development, there were two primary languages considered: Python and C/C++. I have prior experience with both, so I made the decision based on suitability for integrating ML with FHE workflows. Python was chosen due to its strong ecosystem for both AI and FHE development. Python offers actively maintained libraries such as scikit-learn, PyTorch, and TensorFlow for ML, alongside purpose-built FHE libraries like Concrete-ML as well TenSEAL and PySEAL which are open-source libraries that are a Python wrapper of Microsoft SEAL open-source homomorphic encryption library. These libraries are not only more accessible via Python APIs but also backed by strong documentation and active community support. Python also enables rapid prototyping and testing, which is essential for iterative design in a research-orientated project, especially since the project's goal is to evaluate feasibility. Its dynamic typing and high-level abstractions allow quicker experimentation with model architectures, encryption parameters, and benchmarking setups compared to lower-level libraries. C/C++ offers better raw performance and is used as the underlying implementation in most FHE libraries; however, it was deemed less suitable for the project's development phase. FHE pipelines in C/C++ require significantly more boilerplate and cryptographic configuration, making experimentation slower and more error prone. Additionally, many Python-based libraries act as wrappers over performant C/C++ code.

For web application development, HTML, CSS, and JavaScript were used to build the frontend interface, providing the structure, styling, and interactivity of the system. On the backend, Node.js (v22.8.0) was employed with the Express.js framework to handle HTTP requests, server static frontend files, and expose REST endpoints for interacting with the

ML/FHE pipelines. Additionally, Multer middleware was integrated to support both disk-based and in-memory file uploads, enabling the server to efficiently process plaintext images as well as encrypted payloads. This allowed the web application to remain lightweight, responsive, and capable of streaming results in real time from the Python-based pipelines. The web application was developed with the intention of it being an educational and exploratory way of presenting an end-to-end ML service. While alternative backends such as Python Flask or Django could have been used, Node.js was chosen for its asynchronous, event-driven mode, which integrates well with streaming logs and real-time feedback from ML pipelines to the client. This choice minimized overhead and aligned with the project's demonstration goals, even though a production system might demand a more feature or secure web framework.

D. Libraries

To evaluate the feasibility of deploying an FHE-based ML model in a real-world setting, several FHE and ML libraries are being reviewed and used. The goal is to assess how well these tools support encrypted ML inference and whether they are mature enough for reproducible and practical deployment.

Scikit-learn (v1.5.0) is a lightweight and well-documented Python library for classical ML models, such as decision trees, k neighbors, logistic and linear regression. It is ideal for prototyping ML workflows and understanding model performance in plaintext. While it lacks built-in support for encrypted data, it serves as the baseline framework for model development and benchmarking. Many FHE libraries use models initially trained in scikit-learn, converting them to FHE formats. During development it was used to prototype models quickly in plaintext before evaluating the feasibility of encrypted inference, as well as assessing the capabilities of the FHE libraries being evaluated.

TensorFlow (v2.16.2) and **PyTorch** (v2.7.1) are deep learning frameworks that support more complex ML architectures, including neural networks and quantization-aware training. TensorFlow provides model quantization and export tools, while PyTorch offers tight integration with TenSEAL. While both are more complex than scikit-learn, they become necessary when experimenting with advanced FHE-compatible models, particularly in scenarios requiring quantized inference or encrypted tensor operations. For ML pipeline development, PyTorch was chosen over TensorFlow due to its tighter integration with TenSEAL, simplifying the transition from plaintext to encrypted models and inference.

Concrete-ML (v1.9.0) is the most production-focused FHE-ML library reviewed. Built on the TFHE scheme, it supports both leveled and programmable bootstrapped operations, enabling the encrypted evaluation of arbitrary functions, including non-linear activations. Concrete-ML includes a compiler that transforms trained scikit-learn models into FHE circuits. Its automated quantization, deployment-ready pipeline, and client-server architecture make it a strong candidate for real-world applications. However, its support is mostly limited to classical ML models and quantized approximations, so it may not generalize well to deep learning without trade-offs in precision or complexity. While not adopted for pipeline development, Concrete-ML's approach informed the evaluation of and approach of the developed pipeline and introduction into Fully and Leveled homomorphic encryption implementation.

TenSEAL (v0.3.16) wraps Microsoft SEAL, and targets encrypted inference using BFV and CKKS scheme. It integrates with PyTorch and supports SIMD-style encrypted tensor operations, such as dot products and matrix multiplications. TenSEAL is highly suitable for real-valued encrypted inference where the model structure is known and bounded in depth. It avoids bootstrapping, relying instead on leveled FHE. This makes it ideal for secure, low-latency inference applications, where data privacy is paramount, though it requires manual tuning of noise budgets and does not support encrypted training. TenSEAL forms the project's encrypted inference pipeline, providing the practical balance between usability, PyTorch compatibility, and support for CKKS operations.

PySEAL (v2.2) is a low-level Python wrapper over Microsoft SEAL. It exposes the internals of the SEAL library for maximum flexibility in cryptographic configuration. PySEAL supports BFV and CKKS schemes, and while it lacks high-level tensor abstractions, it allows full control over encryption parameters and noise growth. This makes it highly suitable for benchmarking and cryptographic evaluation rather than rapid prototyping or deployment. It is especially useful for validating how different FHE schemes perform under real-world workloads and comparing the precision, ciphertext size, and runtime overhead trade-offs. PySEAL was used during research to assess modern FHE libraries and components of the SEAL library, however its lack of abstractions and preset design made it less suited for the exploratory and iterative approach being performed.

E. Hardware

All implementation and evaluation were conducted and executed on a MacBook M3 Pro. System and software specifications were as follows: Chip: Apple M3 Pro, CPU: 11-core (five performance and six efficiency), GPU: 14-core, Neural Engine: 16-core, Memory: 18 GB, Python Version: 3.10.18, Conda Version: 24.11.3, NPM Version: 10.9.2.

IV. Design and Implementation

A. Sustainability Considerations

One consideration for this project was ensuring that the exploration of FHE and ML pipelines was both sustainable and reproducible. From the outset, parameterization was emphasized through a dedicated configuration file, enabling reproducible experimentation without hardcoding values. This approach ensured that future research could build upon the project by adjusting hyperparameters, dataset sizes, or encryption settings without re-engineering the codebase. In terms of computational sustainability, trade-offs between security, efficiency, and resource consumption were carefully weighed. CKKS contexts were tuned to balance encryption precision with GPU/CPU overheads, avoiding configurations that would exhaust memory or incur infeasible runtimes. Where possible,

dataset sizes and input resolutions were down sampled to ensure experiments could run iteratively without computational costs. These considerations aligned with the broader research goal of making encrypted ML pipelines accessible and educational, not just proof-of-concept.

B. FHE Library Research and Evaluation

Initial research focused on three candidate FHE libraries; Concrete-ML, PySEAL, and TenSEAL. Each was evaluated through small-scale implementations measuring accuracy, execution time, and architectural compatibility.

Concrete-ML was evaluated using six model architectures (decision trees, k-nearest neighbors, linear regression, logistic regression, neural networks, random forest) across dataset sizes ranging from 500 to 10,000 samples. Classification and regression metrics (e.g., accuracy, precision, mean squared error (MSE), mean absolute error (MAE)) were computed for both cleartext and FHE execution. Linear and logistic regression models achieved 100% accuracy across all dataset sizes, demonstrating Concrete-ML's capabilities for shallow models where TFHE's programmable bootstrapping evaluates low-depth circuits efficiently. However, decision trees showed accuracy degradation beyond 10,000 samples and neural networks struggled with larger datasets, revealing scalability limitations regarding circuit depth and constraints in Concrete-ML's quantization pipeline, indicating that it is unsuitable for deep learning architectures.

Concrete-ML was tested with multiple scikit-learn based architectures, including decision trees, k-nearest neighbors, linear and logistic regression, neural networks, and random forests. Performance was assessed using evaluation metrics such as Mean Squared Error (MSE), Mean Absolute Error (MAE), R^2, Mean Absolute Percentage Error (MAPE), and Explained Variance Score (EVS). Execution time, input scaling, and model compile time were also measured. Results demonstrated scalability challenges, such as decision tree classifiers degraded in accuracy beyond 10,000 samples, while linear and logistic regression consistently achieved 100% accuracy across all tested sample ranges. Neural networks and random forests showed similar struggles with larger datasets, highlighting limitations for deep learning under Concrete-ML. TenSEAL was evaluated next, following similar scikit-learn protocols but with limited accuracy on simple sklearn models. However, subsequent testing on a lightweight PyTorch CNN (MNIST classification with one convolutional and two linear layers) demonstrated significantly higher performance. With FHE-compatible polynomial activations (squares), the encrypted model achieved 98% accuracy on 10,000 test samples. This highlighted TenSEAL's advantage for torch-based encrypted deep learning, making it the most suitable library for this project. PySEAL was also examined but deprioritized due to its production-oriented focus on integer arithmetic and limited support for complex numbers. Setup was challenging, and with the library archived, its exploration potential was limited. TenSEAL was selected as the primary FHE library, with Concrete-ML retained for benchmarking against sklearn-based models.

C. ML Pipeline Plan and Design

With TenSEAL selected, the ML pipeline design focused on supporting both classification and regression workflows in plaintext and encrypted domains. The guiding requirements were (1) reproducibility through configuration files for each pipeline, (2) exploration, where pipelines should be easy to fine tune both model and FHE parameters, and (3) evaluation, where the integration of metrics, visualizations and logging for evaluating the pipeline should be the focus and core component. Three pipeline use cases were defined; (1) MNIST Classification, which is a CNN-based digit recognition model, (2) Face Detection, which is a CNN-based bounding box regression model, and (3) Border Generation, which is a CNN image-to-image regression model.

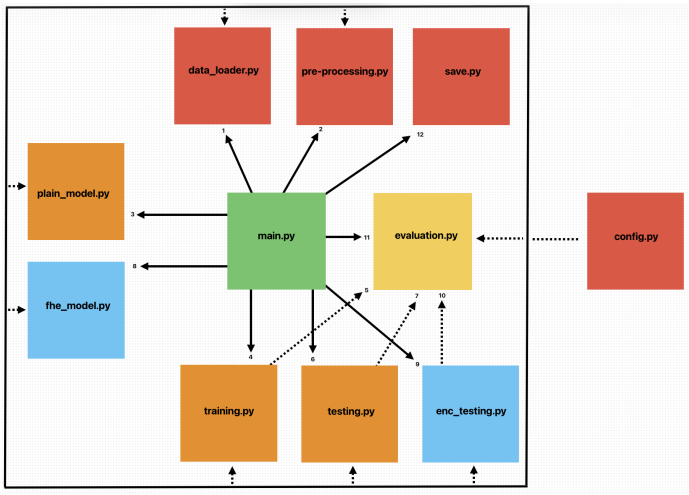

Fig. 5. CRC/UML Diagram for ML+FHE Pipelines

To formalize the pipeline, CRC/UML diagrams (figure 5) were developed in Freeform (v4.0). These diagrams clarified the responsibilities of each class; data loader, preprocessing, model, training, testing, evaluation, FHE model, encrypted testing, and save as well as their interactions. This design step ensured modular development, easy debugging, and alignment with standard software engineering practices. Pipeline was also designed to be executed in two different modes, testing and final. Testing allows outputs to be presented to the user throughout pipeline runtime and provides more terminal based information throughout its runtime. Final mode does not provide these aspects; however, it saves the model at the end of the runtime to then be used with the web application.

D. Evaluation Implementation

Evaluation was planned and implemented as a dedicated layer in the pipeline. For classification, metrics included accuracy, precision, recall, F1, and confusion matrix visualizations. For regression, MSE, MAPE, R^2, EVS, residual plots, and loss curves were generated. Resource monitoring (CPU, GPU, RAM) was integrated using psutil and torch.cuda, allowing comparisons of plaintext versus. encrypted inference efficiency. To support tracking across multiple runs, results were stored in continuously updated .csv logs, and automated

PDF reports were generated using ReportLab. These reports included plots, model predictions, CKKS parameter summaries, and runtime logs. By automating evaluation outputs, the pipeline ensured consistency, transparency, and traceability across all experiments.

E. Data Loader and Preprocessing Implementation

The data loader and preprocessing components were designed to be modular and reusable across all pipelines. Each dataset followed a consistent workflow: loading and normalization, reshaping into the required tensor format, visualization (for testing), optional data augmentation to improve generalization, and splitting into training (75%), validation (15%), and test (10%) sets with fixed random seeds to ensure reproducibility. The preprocessed data was then converted into PyTorch tensors and returned to the main pipeline class. For the number detector model, a sample of 2000 images were loaded, normalized, and reshaped into $(N, 96, 96, 3)$, where N is the dataset size, and 3 channels represent RGB values. Since the dataset provides its own train/test splits but the project required explicit control, all images were concatenated before preprocessing. Augmentation techniques (flip, rotation, zoom) were applied to increase robustness.

For the face detection model, a sample of 1600 images from the Human Faces dataset were prepared and reshaped into $(N, 196, 196, 3)$. As the dataset lacked labels, bounding box

TABLE III
PLAINTEXT PIPELINE MODEL AND TRAINING CONFIGURATION [18]

Task	Input Size	Architecture	Output	Training Setup
Number Detector	1×28×28 (greyscale)	3 Conv2d: 1→8→16→32 Stride: 1, Padding: 1, Kernel: 3 BatchNorm, ReLU 2×2 pooling, Dropout: 0.25 Flatten: 32×3×3→288 2 Linear: 288→64→10	10 Classes	Optimiser: Adam Learning Rate: 0.001 Epochs: 50 Batch Size: 32 Loss: Cross Entropy Data Split: 75/15/10 Augmentation: True
Face Detection	3×192×192 (RGB)	3 Conv2d: 3→32→64→128 Padding: 1, Kernel: 3 ReLU, 2×2 pooling Flatten: 128×24×24→73728 2 Linear: 73728→256→4	Bounding boxes: [cx, cy, bw, bh]	Optimiser: Adam Learning Rate: 0.001 Epochs: 50 Batch Size: 16 Loss: Mean Squared Error Data Split: 75/15/10 Augmentation: False
Border Generation	1×192×192 (greyscale)	2 Conv2d: 1→8→16 Padding: 1, Kernel: 3 SqAct→192×96×48 2 ConvTransP2d: 16→8→1 SqAct→48×96×192	Bordered Image	Optimiser: Adam Learning Rate: 0.001 Epochs: 50 Batch Size: 8 Loss: Mean Squared Error Data Split: 75/15/10 Augmentation: False

TABLE IV
FHE PIPELINE MODEL, TRAINING, AND ENCRYPTION CONFIGURATION

Task	Input/Output	Architecture	Training Setup	Encryption Setup
Number Detector	1×28×28 / 10 Classes	Conv2d: 1→12 Stride: 3, Kernel: 7 SqAct, Dropout: 0.3 Flatten: 12×8×8→768 2 Linear: 768→128→10	Optimiser: Adam LR: 0.0005 Epochs: 75 Batch: 16 Loss: Cross Entropy Split: 75/15/9/1 Augment: Yes	CKKS Scheme Poly Modulus: 8192 Scale Bits: 26 Coeff Mod Size: 8 Keys: 31
Face Detection	3×128×128 / [cx, cy, bw, bh]	Conv2d: 3→1 Stride: 3, Kernel: 1 SqAct, Dropout: 0.3 Flatten: 64×64→4096 2 Linear: 4096→512→4	Optimiser: Adam LR: 0.001 Epochs: 100 Batch: 16 Loss: MSE Split: 75/15/9/1 Augment: No	CKKS Scheme Poly Modulus: 8192 Scale Bits: 26 Coeff Mod Size: 8 Keys: 31
Border Detection	1×32×32 / Bordered Image	Conv2d: 1→4 (Stride: 2) SqAct Conv2d: 4→1	Optimiser: Adam LR: 0.001 Epochs: 150 Batch: 16 Loss: MSE Split: 75/15/9/1 Augment: No	CKKS Scheme Poly Modulus: 32768 Scale Bits: 40 Coeff Mod Size: 6 Keys: 60

ground truth values $[cx, cy, bw, bh]$ were generated using the face recognition library. These labels were stored alongside the images and returned as arrays for further preprocessing. For the border generation model, a sample of 2500 images consisting of human faces were used, reshaped into $(N, 192, 192, 1)$. Unlike the face detection task, the target output was synthetically generated by applying a border to the input image, effectively creating an image-to-image regression dataset. To reduce computational cost under FHE constraints, only a single grayscale channel was retained. Across all datasets, preprocessing visualized input/output pairs during testing mode and applied consistent train, validation, and test splits with fixed random seeds. This standardization allowed each pipeline to reuse the same structural design while adapting only to dataset specific requirements.

F. Plaintext Pipeline Model Implementation

All plaintext pipelines were implemented in PyTorch following a consistent workflow (Table 3). Each model included training validation, and testing classes handling forward passes, loss computation, backpropagation, and optimizer updates. The Adam optimizer was used collectively with task-specific learning rates. Training ran for 50 epochs with batch sizes varying by dataset and task complexity. Classification used CrossEntropyLoss while regression tasks used MSE. Validation ran in evaluation mode without gradient tracking, logging ground truth, predictions, timings, and losses for visualization and reporting. Testing evaluated unseen data, generating performance metrics and visual comparisons. For each of the three models (Number Detector, Face Detection, Border Generation), input size, model architecture, model outputs, and training setup are specified in Table 3.

G. FHE Pipeline Model Implementation

All FHE pipelines follow a dual-architecture approach where plaintext models are trained conventionally in PyTorch, then converted to FHE-compatible versions using TenSEAL's CKKS scheme (Table 4). Each pipeline uses square activation (x^2) instead of ReLU to maintain polynomial compatibility required for homomorphic encryption. Training configurations, architectures, and encryption parameters are detailed in Table 3.

The FHE MNIST pipeline processes 1x28x28 grayscale images using a simplified CNN with one convolutional layer producing 12x8x8 features. After square activation and dropout, features are flattened to 768 dimensions and passed through two fully connected layers. The FHE model extracts trained weights and reimplements forward passes using encrypted operations. Inputs are encoded with ts.im2col_encoding, convolved using conv2d_im2col, and channels are packed with CKKSVector.pack_vectors. Square activations replace ReLU, and matrix multiplications implement fully connected layers while maintaining encryption throughout. The CKKS context uses polynomial modulus 8192, coefficient bits [31,26,26,26,26,26,26,31], and bits scale 26, with Galois keys enabling encrypted rotations for convolutions.

The FHE Face Detection pipeline regresses bounding boxes [cx, cy, bw, bh] from 3x128x128 RGB inputs. A single convolutional layer down samples to 1x64x64, applying square activation before flattening to 4096 features. Two fully connected layers with dropout produce bounding box coordinates. The FHE implementation processes each RGB channel separately with conv_im2col, summing outputs with bias terms. Encrypted activations pass through fully connected layers, maintaining encryption until decryption for evaluation. The CKKS context mirrors MNIST parameters, ensuring precision and security balance. Visual outputs overlay predicted and ground truth boxes for interpretability.

The FHE Border Generation pipeline processes 1x32x32 grayscale images through a lightweight autoencoder with two convolutions. The first convolution expands to 4 channels with square activation, while the second compresses back to 1 channel, preserving spatial dimensions. The FHE model extracts weights and implements encrypted convolutions using conv2d_im2col with square activations replacing nonlinearities. Encrypted feature maps are recombined through the second convolution, producing encrypted image transformations. Due to increased computational depth, this pipeline uses polynomial modulus 32768, coefficient bits [60,40,40,40,40,60], and bits scale 40 for higher precision.

The save class is called when the pipeline is in final mode and saves the trained mode state dictionary to disk. The method generates a timestamped filename and stores the model in the corresponding model directory associated with the pipeline directory, ensuring that it is identifiable based on the training run. This ensures reproducibility and tracking later during evaluation.

H. Web Application Planning and Design

The web application serves as both a demonstration tool and a validation service where plaintext and encrypted ML models are compared against each other. The web application contains all three models; MNIST, Face and Border, where it provides a clear user interface with logs and visualization. Wireframes were developed (Figure 6, 7) to define workflows; user input $\rightarrow$ encryption (trusted server) $\rightarrow$ inference (untrusted server) $\rightarrow$ decryption (trusted server) $\rightarrow$ output visualization. This planning ensured usability and security alignment.

I. Web Application Implementation

The web application for the PrivML-FHE framework was structured around a modular separation of concerns, with distinct frontend, backend, and helper components working together to enable both plaintext and encrypted workflows. Web application is implemented and intended for localhost/loopback use only.

The client interface was implemented using separate HTML, CSS, and JavaScript files. Each HTML page (start.html, mnist.html, face.html, border.html) defined the layout and workflow for the corresponding model, including navigation menus, canvases for image input or drawing, encryption/decryption controls, and result display areas. Styling was centralized in style.css, ensuring consistent appearance across

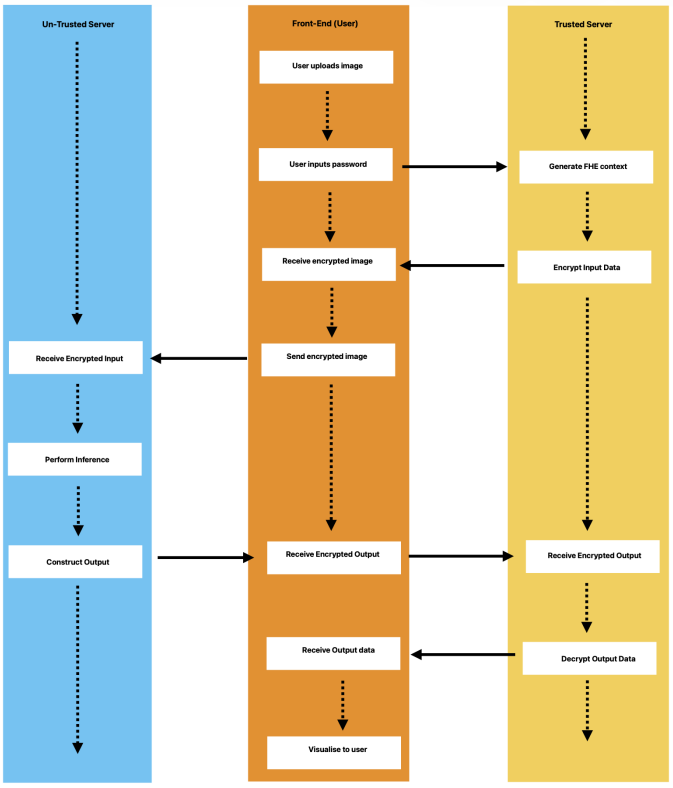

Fig. 6. Untrusted and trusted server client interaction

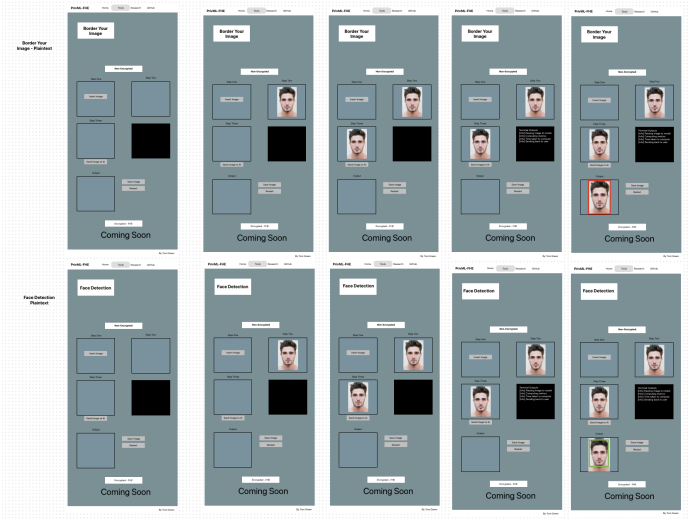

Fig. 7. Initial Designs for web application model pages

sections with responsive design, section chips, panels, toolbars, and interactive buttons. Interactivity was managed by nav.js, which handled navigation, button clicks, and asynchronous communication with the backend servers. For each pipeline, workflows were divided into two JavaScript modules: one for plaintext inference and one for encrypted inference. These scripts managed the upload of images, user drawing, encryption password entry, visualization of "manufactured encryption" effects, and the streaming of outputs from the backend.

The untrusted server was hosted on port 3000 and was implemented with Node.js and Express.js and served as the orchestrator for frontend–backend interactions. It hosted static frontend assets, while managing pipeline-specific jobs (jobsMnist, jobsFace, jobsBorder, and their FHE equivalents) via in-memory maps. For plaintext tasks, image uploads were handled with Multer, which stored inputs either on disk or in memory. Encrypted payloads (up to 500×1024×1024 in size) were serialized and passed securely through the pipeline. Each client request initiated a POST to /api/[type]/upload (with [type] being mnist, face, border, etc.), generating a unique UUID jobId used to track file paths and outputs. Model inference was performed by spawning a Python process with child_process.spawn, passing file paths and Python environment parameters. Standard output and errors were streamed back to the client in real time using Server-Sent Events (SSE), and final predictions were returned as JSON or base64-encoded images. For encrypted workflows, payloads contained serialized ciphertexts and TenSEAL contexts. These were validated, written to JSON files, and forwarded to the appropriate encrypted model script (e.g., fhe_mnist.py). The untrusted server could not decrypt payloads, thereby ensuring security; all temporary artifacts were deleted after execution.

The trusted server was hosted on port 4000 and represented the secure component of the architecture, built on Node.js and Express.js but restricted to encryption, decryption, and context management. It exposed REST endpoints (/api/fhe_[type]_encrypt and /api/fhe_[type]/_decrypt) to handle sensitive operations. Requests were relayed to Python helper scripts (e.g., fhe_mnist_client_helper.py), which performed cryptographic transformations using TenSEAL. Unlike the untrusted server, the trusted backend had access to private CKKS keys, enabling both encryption of raw user inputs and decryption of encrypted outputs. Inputs such as raw image data, convolution parameters, and user-provided passwords were validated, serialized, and processed by Python helpers, which returned JSON results containing ciphertexts and serialized CKKS contexts. Similarly, decrypted outputs were reshaped into usable formats and returned securely to the client. CORS was enabled to allow requests from the untrusted server, which acted as the main client interface.

The Python client helper scripts served as the FHE engine of the trusted server. At startup, each helper constructed a TenSEAL CKKS context using pipeline-specific configurations (polynomial modulus degree, coefficient modulus bit sizes, Galois keys). A secret key was generated to allow encryption and decryption within the trusted domain. For encryption, helpers ingested preprocessed input data, normalized and reshaped it, then encoded and encrypted it into a CKKS ciphertext object using ts.ckks_vector. These ciphertexts, serialized with the public portions of the context, were returned to the untrusted server for homomorphic computation. For decryption, helpers reconstructed ciphertexts and contexts, applied decrypt(), and returned recovered predictions in plaintext. This modular design allowed the untrusted server to handle homomorphic inference without direct access to secret keys.

Plaintext inference was handled by Python scripts (mnist.py, face.py, border.py), spawned by the untrusted server. Each script reconstructed user inputs (e.g., resizing images via

PIL), loaded the appropriate model checkpoint with torch.load, restored architectures into evaluation mode, and performed inference under torch.no_grad(). Execution time was logged, and results were returned to the client as structured JSON or images. Continuous structured logging was sent back via SSE for monitoring. Failure cases were gracefully managed, returning structured error objects.

Encrypted inference extended the plaintext pipeline through FHE wrappers (fhe_mnist.py, fhe_face.py, fhe_border.py). Instead of tensors, the inputs were serialized ciphertexts and contexts received from the trusted server. These were reconstructed using ts.context_from and ts.ckks_vector_from. For multi-channel data, each channel was restored separately into encrypted vectors. The plaintext PyTorch model was loaded as before, then wrapped into an FHE model counterpart that redefined operations (convolutions, multiplications, activations) over encrypted data using TenSEAL's CKKS scheme. Convolutions were implemented with encrypted sliding windows (im2col_encoding), and outputs remained encrypted throughout. Predictions were serialized back into ciphertexts with metadata (e.g., inference time, kernel shapes, strides, and processing completion flags), ensuring no plaintext leakage. Logs and error messages were returned securely, with failed jobs marked as complete without revealing sensitive data.

J. User Testing for Web Application

User testing was performed in relation to two articles [25][26] covering the system usability scale or SUS, developed by John Brooke in 1986. SUS is a standard way to assess a product's overall user-friendliness and to evaluate the usability of digital products in websites and apps, which was. SUS is helpful as it enhances user-centred evaluation, provides quantitative measurement, offers speed and simplicity, facilitates iterative design, supports decision-making, enhances collaboration, promotes consistency, encourages user feedback, and aids in resource allocation.

To ensure that user testing followed the SUS model, I first created a Google Forms, which had the SUS form that users would fill out at the end of user testing the application. This contained ten questions and 3 optional written areas, ensuring that it was simple and brief and an efficient way of collecting usability data. The 10 questions followed closely with the SUS questionnaire laid out in [25] and the 3 optional areas allowed the user the ability to provide positive, negative, and improvement written feedback. I next created a user testing questions to ask each person while using the system that covered all aspects of what it provided in terms of user interaction, which spanned over 28 questions. This allowed and ensured that consistency between each user test was acknowledged and maintained but also allowed the user to freely use the system to find bugs and errors that were potentially not identified.

User testing began in the third week of web application development and continued after the initial implementation was completed. The web application achieved SUS user ratings from 77.5-95 an average SUS score of 82.97, indicating great usability. The high score indicates that users found components of the application intuitive and straight-forward. A total of 15 users participated, each engaging in the application for a set duration of 10 minutes. Overall feedback was positive, each user engaging with the application for a set duration of 10 minutes. Overall feedback was positive, particularly in terms of ease of use, which received scores consistently between 4 and 5. Similarly, the design and UI also rated highly within the same range. However, some early testers struggled to understand the core concept of the application. As a result, lower scores were given for consistency and performance. One key issue identified was with the number detector model, which initially lacked generalization capabilities and often failed to classify numbers correctly. Additional feedback highlighted the difficulty users faced in understanding what was happening duration interaction, especially due to the lack of clear guidance on when to proceed to the next step. Suggestions included reworking the layout to a more intuitive top-down structure instead of side-by-side boxes and ensuring consistency between the design of these pages and the rest of the web application. A specific recommendation was to rename the heading Terminal Outputs to Log Outputs, to better reflect the output shown and improve user understanding of the FHE model's behaviour. This feedback directly influenced improvements in both the user experience and the application design. The number detector model has since been significantly enhanced, showing far better performance than the early versions. The final iteration of the web application presents recommended design changes (Figure 8).

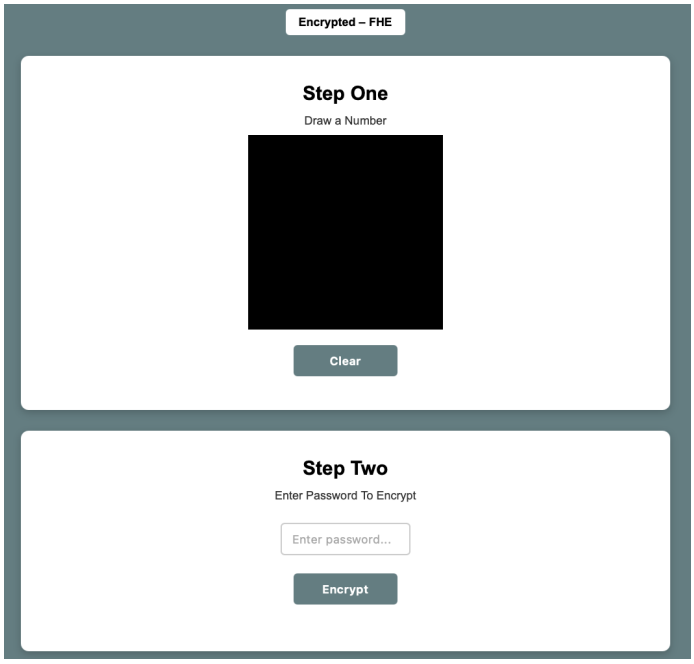

Fig. 8. Updated Web Application Design

V. Results and Performance Metrics

Evaluation of the ML pipelines followed a systematic approach designed to assess model performance across both plaintext and encrypted domains. Hyperparameter tuning was performed extensively alongside evaluation to identify optimal

configurations that balanced accuracy, computational feasibility, and FHE compatibility. The evaluation framework was structured to ensure reproducibility, transparency, and traceability across experimental runs. Each pipeline maintained a consistent modular structure which enabled direct comparison between plaintext and encrypted model performance while isolating the impact of homomorphic encryption on inference accuracy and computational overhead. Model tuning was performed primarily on the encrypted workflows as the plaintext models were developed during initial sprints to establish baseline architectures. Once plaintext models demonstrated satisfactory generalization, they were adapted for FHE compatibility and explored hyperparameter tuning to match results of their counterpart. Model tuning and results evaluation were only performed on the encrypted workflow. Implementation of the non-encrypted pipeline was performed over a large sprint as this was initial model development exploring multiple loss functions and optimization functions and model development stages to ensure that the model generalized the best. This was then adapted once final plaintext models were defined for the encrypted pipeline.

Hyperparameter tuning for model pipelines included nine optimization functions (Adam, AdamW, Adamax, Radam, Adadelta, Adagrad, RMSprop, SGD, RProp), nine epoch configurations (30, 50, 75, 100, 150, 200, 300, 400, 500), six learning rates (0.0001, 0.0005, 0.001, 0.005, 0.01, 0.05), four batch sizes (16, 32, 64, 128), four loss functions (MSELoss, CrossEntropyLoss, L1Loss, KLDivLoss), six dataset sizes (500, 1600, 3000, 5000, 7000, 10000), five random seeds (42, 123, 500, 999, 1337), and four dropout rates (0.1, 0.3, 0.5, 0.6). In total, approximately 62 distinct experimental configurations were evaluated per encrypted mode, resulting in over 150 training runs across all three pipelines.

Automated evaluation scripts generated structured outputs for each experimental run, with metrics appended to a continuous CSV file and model predictions and auto generated plots to PDF reports. These outputs provided both insightful and visual evidence of model performance, enabling comparison across all configurations. All evaluation artifacts can be accessed in the output's directory of each pipeline, with PDF reports from model runs accessible in a shared link in the README.

A. Border Model Evaluation

The border generation pipeline was evaluated across 51 experimental configurations using regression metrics. Model 26 achieved optimal performance with a validation MSE of 0.0382 and encrypted MSE of 1.5320, demonstrating the best balance between plaintext training stability and encrypted inference accuracy. This configuration utilized a batch size of 16, which reduced memory pressure in the encrypted domain while enabling smoother gradient computations compared to larger batch sizes, such as 32, 64, and 128, which degraded performance in this pipeline due to over-smoothing. The Adam optimizer with a learning rate of 0.001 proved most effective for the squared activation function used in this FHE-compatible architecture. Model 7 (baseline Adam configuration) achieved validation MSE of 0.0367 but encrypted MSE of 260.5482, which is worse than Model 26. Higher learning rates (0.01, 0.05) destabilized training, while smaller rates (0.0001, 0.0005) converged too slowly. Epoch analysis revealed that 150 epochs (Model 14: validation MSE 0.0367, encrypted MSE 265.3735) provided optimal generalization, as 50 epochs (Model 11: encrypted MSE: 249.5713) caused underfitting and 300 epochs (Model 16: encrypted MSE 292.5049) led to overfitting. The Adamax optimizer (Model 4: encrypted MSE 2753162) performed competitively but remained less stable than Adam. MSELoss was consistently superior for this regression task, with the model 26 achieving MAE of 1.2248 and inference time of 3.9952 seconds on encrypted data using CKKS scheme (poly modulus degree 32768, bit scale 40).

TABLE V
FHE BORDER GENERATION MODEL EVALUATION RESULTS[1]

Model	Dataset	MSE	MAE	Time (s)
Model 26	Validation	0.0382	0.1257	6.6545
	Testing	0.0445	0.1508	0.0022
	Encrypted	1.532	1.2248	3.9952
Model 7	Validation	0.0367	0.1289	7.0296
	Testing	0.0400	0.1409	0.0005
	Encrypted	260.5483	16.1403	4.0168
Model 14	Validation	0.0367	0.1243	21.2513
	Testing	0.0395	0.1339	0.0004
	Encrypted	265.3735	16.2891	4.0281
Model 11	Validation	0.0366	0.1266	7.3240
	Testing	0.0399	0.1382	0.0004
	Encrypted	249.5713	15.7966	4.0106
Model 16	Validation	0.0367	0.1239	41.9716
	Testing	0.0398	0.1346	0.0005
	Encrypted	292.5049	17.1016	4.0002
Model 4	Validation	0.0368	0.1276	6.9325
	Testing	0.0402	0.1400	0.0023
	Encrypted	275.3161	16.5915	3.9912
Model 51	Validation	0.0376	0.1307	49.6738
	Testing	0.0541	0.1442	0.0004
	Encrypted	270.2084	16.4367	3.7113

The final border detection model (Model 51) was selected based on its training configuration outlined in Table 4 (Section G). Evaluation of the model's performance reveals a validation MSE of 0.0376 and test MSE of 0.0541, demonstrating consistent generalization across plaintext evaluation sets. However, encrypted inference produced an MSE of 270.2084, representing a significant degradation compared to validation performance. This increase reflects the accumulated noise and approximation errors inherent to CKKS encryption when operating on 32x32 image reconstruction tasks over more than one convolutional layers with high polynomial parameters. Despite this degradation, the encrypted MAE of 16.4367 indicates that per-pixel errors are bounded, suggesting the model maintains structural fidelity under encryption. The inference time of 3.7113 seconds per encrypted sample demonstrates computational feasibility for non-real-time applications, while

[1]Validation time refers to the total time required to evaluate the full validation set, while testing and encrypted times represent the average time taken to evaluate a single sample under standard and encrypted inference, respectively.

validation time of 49.6738 seconds confirms the efficiency of plaintext evaluation.

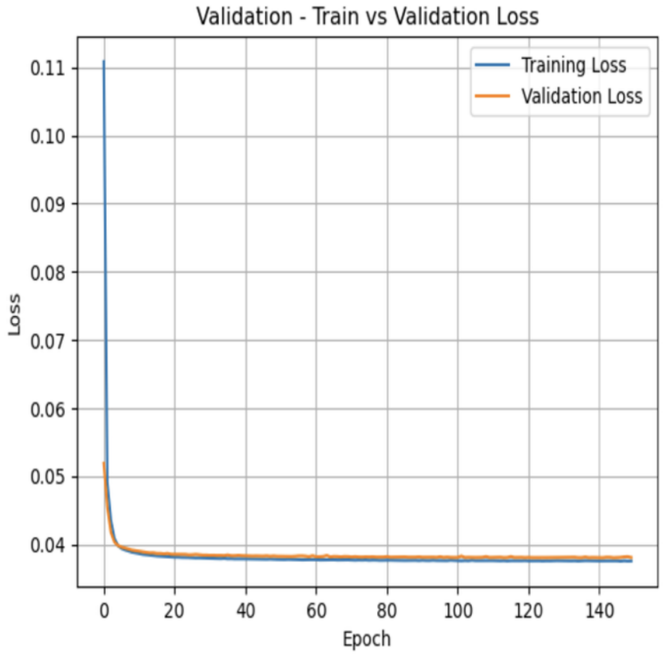

Fig. 9. FHE Final border model training and validation losses

Training and validation loss curves (Figure 9) provide critical evidence of model selection, showing smooth convergence over 150 epochs with both curves stabilizing around 0.038 after approximately 20 epochs. The tracking of validation loss through the entire training period indicates strong generalization without overfitting or underfitting, validating the choice of 150 epochs and learning rate 0.001. The early stabilization suggests that the model efficiently learned the border generation tasks within the first 20 epochs, with subsequent iterations refining the weights without introducing variance. Runtime profiling during encrypted inference revealed CPU usage between 1.6-5.0% and RAM usage of 6.6GB (73% of 18GB total), showing computational power needed to perform operations. The encryption key size of 1.31MB and global scale parameter of 2^{40} enabled sufficient precision for the image reconstruction task, maintaining decryption stability across all encrypted test samples.

B. Face Detection Evaluation

The face detection pipeline was evaluated using regression metrics across 62 experimental configurations. Model 58 achieved optimal performance with validation MSE of 0.0100 and encrypted MSE of 0.0021, representing a 79% improvement in encrypted inference accuracy. This configuration achieved MAE of 0.0343, MAPE of 0.0609, and inference time of 44.3574 seconds. Model 39 performed comparably with validation MSE of 0.0064 and encrypted MSE of 0.0017 (73% improvement), achieving an R^2 score of 0.7301 and EVS of 0.8200, indicating stronger explained variance despite slightly higher validation loss. The encrypted configuration utilized CKKS with polynomial modulus degree of 8192 and bit scale 26, which provided more efficient computation compared to the border model. The AdamW optimizer (Model 4: validation MSE 0.0087, encrypted MSE 0.0108) demonstrated generalization through weight decay regularization, outperforming other Adam functions. Batch size analysis showed that 32 samples (Model 28: encrypted MSE 0.0094) provided optimal stability without over-smoothing gradients, while learning rate 0.001 (Model 22: encrypted MSE 0.0282) proved most effective for Adam optimizers. Hidden dimension scaling showed size 512 (Model 48: encrypted MSE 0.0224) significantly improved feature representation capacity over the 256-unit baseline, while 1024 units (Model 49: encrypted MSE 0.0145) led to overfitting despite lower encrypted error. The larger 7000-sample dataset enabled successful training of higher capacity architectures validating the importance of dataset scale for complex regression tasks under FHE constraints.

TABLE VI
FHE FACE DETECTION MODEL EVALUATION RESULTS[2]

Model	Dataset	MSE	MAE	Time (s)
Model 58	Validation	0.0100	0.0722	12.6813
	Testing	0.0085	0.0672	0.0672
	Encrypted	0.0021	0.0343	7.3929
Model 39	Validation	0.0064	0.0585	53.5993
	Testing	0.0217	0.0924	0.0012
	Encrypted	0.0017	0.0321	9.0462
Model 4	Validation	0.0087	0.0684	14.9787
	Testing	0.0102	0.0741	0.0015
	Encrypted	0.0108	0.0771	9.0290
Model 28	Validation	0.0094	0.0713	12.7245
	Testing	0.0083	0.0675	0.0007
	Encrypted	0.0094	0.0718	7.4514
Model 22	Validation	0.0114	0.0766	13.6932
	Testing	0.0098	0.0786	0.0029
	Encrypted	0.0282	0.1165	7.3645
Model 48	Validation	0.0098	0.0737	17.5839
	Testing	0.0083	0.0727	0.0029
	Encrypted	0.0224	0.1174	7.6992
Model 49	Validation	0.0116	0.0779	26.7562
	Testing	0.0096	0.0728	0.0013
	Encrypted	0.0145	0.0827	8.5531
Model 62	Validation	0.0057	0.0539	266.7120
	Testing	0.0073	0.0685	0.0031
	Encrypted	0.0036	0.0444	9.3931

The final face detection model (Model 62) was selected based on its training configuration outlined in Table 4 (Section G). Evaluation metrics showed an encrypted MSE of 0.0036, which outperformed both validation MSE (0.0057) and test MSE (0.0073), representing a 37% improvement over validation and 51% improvement over test performance. This outcome indicates that the encrypted test subset contained fewer challenging samples or benefitted from the approximate arithmetic properties of CKKs, which may have regularized predictions by smoothing outliers. The encrypted MAE of 0.0444, MAPE of 0.0780, R^2 of 0.5533, and EVS of 0.6946

[2]Validation time refers to the total time required to evaluate the full validation set, while testing and encrypted times represent the average time taken to evaluate a single sample under standard and encrypted inference, respectively.

demonstrate strong predictive performance for bounding box regression, with R^2 indicating the model explains 55% of variance in bounding box coordinates and EVS showing 69% variance is captured by the learned representation. The inference time of 56.3586 seconds over 6 encrypted samples indicate a slightly slower encrypted inference compared to the border model coming in at approximately 9 seconds per sample. However, it remains computationally feasible for processing scenarios where privacy guarantees justify trade-off.

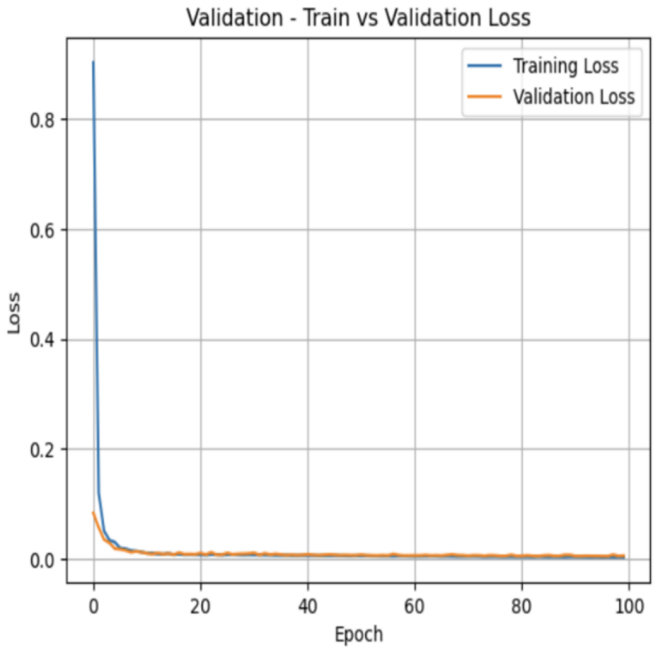

Fig. 10. FHE Final Face final training and validation losses

Training and validation loss curves (Figure 10) show a sharp initial decline within the first 10 epochs before converging smoothly below 0.02, with validation loss tracking training loss closely throughout the full 100 epochs. This behavior confirms optimal convergence without overfitting, validating the selection of 100 epochs and learning rate of 0.001. The early convergence suggests the single 1x1 (kernel) convolutional layer architecture with a 512 hidden dimension provided sufficient representational capacity for the bounding box regression task without requiring deeper layers that would increase multiplicative depth and noise accumulation under CKKS encryption. Runtime usage showed 12.9% CPU usage and RAM usage of 5.95GB (78% of 18GB), demonstrating higher computational load despite smaller encryption key sizes 208KB and lower polynomial modulus degree 8192. This increased CPU utilization reflects the larger input resolution and the high dataset size, confirming that input dimensionality and dataset size drive CPU load more significantly than encryption parameters.

TABLE VII
FHE NUMBER DETECTION MODEL EVALUATION RESULTS[3]

Model	Dataset	Accuracy	Precision	Time (s)
Model 3	Validation	0.9614	0.9610	17.1323
	Testing	0.9569	0.9579	0.0312
	Encrypted	1.0000	1.0000	0.0101
Model 4	Validation	0.9586	0.9580	18.9158
	Testing	0.9597	0.9602	0.0313
	Encrypted	1.0000	1.0000	0.0915
Model 10	Validation	0.9711	0.9710	76.7559
	Testing	0.9676	0.9683	0.0300
	Encrypted	0.9583	0.9500	0.0945
Model 14	Validation	0.9778	0.9770	303.4820
	Testing	0.9685	0.9691	0.0315
	Encrypted	1.0000	1.0000	0.0954
Model 19	Validation	0.9653	0.9650	17.9357
	Testing	0.9620	0.9622	0.0233
	Encrypted	1.0000	1.0000	0.0761
Model 32	Validation	0.9631	0.9620	27.6206
	Testing	0.9620	0.9621	0.0239
	Encrypted	0.9167	0.9467	0.0781
Model 33	Validation	0.9667	0.9660	33.4555
	Testing	0.9657	0.9657	0.0226
	Encrypted	0.9583	0.9800	0.0846
Model 47	Validation	0.9740	0.9740	193.1960
	Testing	0.9700	0.9707	0.0373
	Encrypted	0.9700	0.9698	1.2832

C. Number Detection Evaluation

The MNIST classification pipeline was evaluated using classification metrics across 47 experimental configurations. The classification task provided significantly easier to optimize than regression pipelines, with multiple configurations achieving perfect encrypted accuracy. Model 3 (Adamax optimizer) achieved validation accuracy of 96.14%, test accuracy of 95.69%, and perfect encrypted accuracy of 100.00% with inference time of 4.0576 seconds. Model 4 (Radam optimizer) similarly achieved 95.86% validation accuracy, 95.97% test accuracy, and 100.00% encrypted accuracy (inference time 36.6109 seconds). Model 10 (Adam optimizer) achieved the highest validation accuracy of 97.11% and test accuracy of 96.76%, though encrypted accuracy was slightly lower at 95.83%. Epoch analysis demonstrated that 75 epochs (Model 8) provided optimal convergence, achieving 96.64% validation accuracy, 96.44% test accuracy, and 100.00% encrypted accuracy with F1-score 1.0000. While higher epoch counts (150 epochs, Model 14: validation accuracy 97.78%) marginally improved validation performance, all configurations achieving 100% encrypted accuracy indicated saturation on the encrypted test subset. Learning rate 0.001 (Model 19: validation accuracy 96.53%, encrypted accuracy 100.00%, inference time 30.4739 seconds) proved optimal, balancing convergence speed and generalization. Hidden dimension scaling revealed that 128 units (Model 32: encrypted accuracy 91.67%) and 256 units (Model 33: encrypted accuracy 95.83%) both provided suf-

[3]Validation time refers to the total time required to evaluate the full validation set, while testing and encrypted times represent the average time taken to evaluate a single sample under standard and encrypted inference, respectively.

ficient representational capacity without overfitting. CrossEntropyLoss consistently outperformed alterative loss functions for this multi-class classification objective. The 10000 sample dataset enabled robust generalization with dropout regularization (0.3) effectively controlling overfitting while maintaining FHE compatibility. The CKKS encryption scheme (poly modulus degree 8192, bit scale 26) enabled efficient encrypted inference with minimal accuracy degradation compared to plaintext evaluation.

The final number detector model (Model 47) was selected based on its training configuration outlined in Table 4 (Section G). Evaluation metrics demonstrate good performance with a validation accuracy of 97.40%, test accuracy of 97.00%, and encrypted accuracy of 97.00%. The alignment between test and encrypted accuracy indicates near-zero accuracy degradation from FHE operations, showing the most successful encrypted inference outcome across all three pipelines. The result validates CKKS encryption with polynomial modulus degree 8192 and bit scale 26 which introduces minor approximation error for classification tasks on 28x28 grayscale images. The encrypted precision of 96.98, recall of 96.87% and F1-score of 0.9689 confirm balanced performance across all 10-digit classes, with minimal class bias. The inference time of 513.3167 across all 400 encrypted samples 400 produces an inference time of 1.2 seconds per sample, which represents the highest computational cost among all three final models.

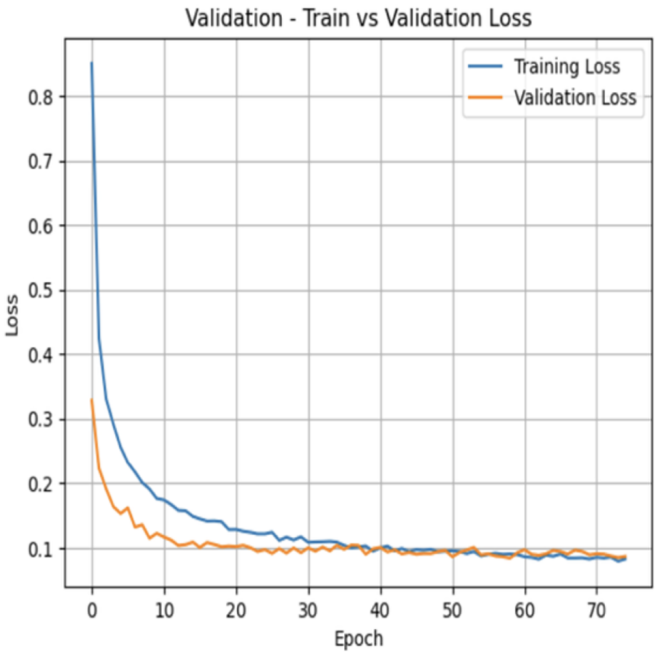

Fig. 11. FHE Final MNIST training and validation losses

Training and validation loss curves (Figure 11) show sharp convergence within the first 10 epochs before stabilizing below 0.1 loss, with validation loss tracking training loss tightly throughout all 75 epochs. This behavior validated the use of 75 epochs and learning rate 0.0005, confirming the model achieved optimal convergence without requiring additional training iterations. The confusion matrix (Figure 12) provides critical evidence for model robustness indicating strong class

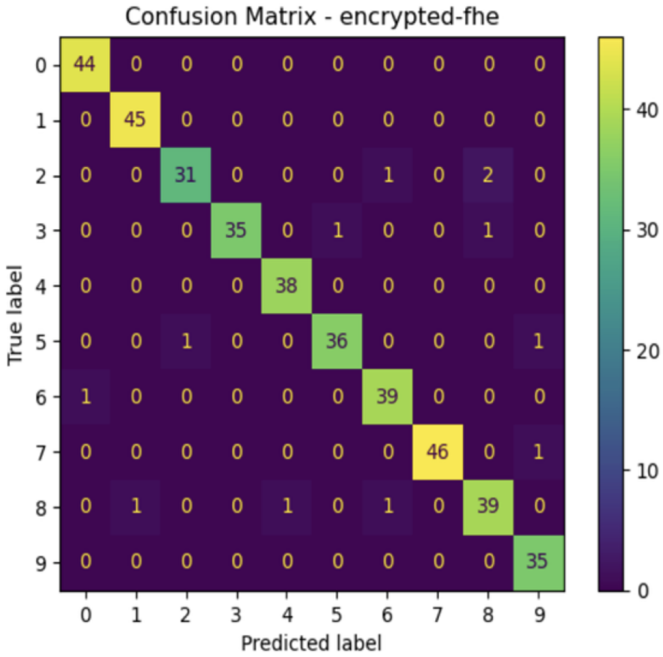

Fig. 12. FHE Final MNIST confusion matrix

accuracy across all 0-9 digits with misclassifications limited to visually similar digit pairs (e.g., 2→8, 4→9), which is consistent with perceptual confusion which is expected. Runtime showed CPU usage between 7.7-11.1% and RAM usage averaging 8.0-8.5GB (63% if 18GB), representing the most balanced resource consumption model.

D. Plaintext and FHE Model Evaluation

TABLE VIII
WEB APPLICATION OUTPUT TIMES FROM PLAINTEXT AND FHE MODELS

Model Type	**Plaintext Time (s)**	**FHE Time (s)**
Number Detector	0.022	1.601
Face Detection	0.020	9.303
Border Generation	0.138	0.263

Table 8 presents web application inference times comparing plaintext pipeline models against FHE equivalents. Face Detection demonstrates the most significant computational overhead, with FHE inference requiring 9.303 seconds compared to 0.020 seconds plaintext. This gap reflects the complexity of encrypting 128×128 RGB inputs and processing through 512-unit hidden layers under CKKS constraints. Number Detector shows a slowdown (0.022s to 1.601s), representing moderate overhead attributed to efficient CKKS parameter tuning (poly modulus 8192, bit scale 26) for 28×28 grayscale inputs. Border Generation exhibits the smallest relative overhead (0.138s to 0.263s), indicating successful architectural optimization through aggressive input dimensionality reduction (32×32) and shallow convolutional depth. These measurements confirm that FHE computational cost correlates strongly with input resolution, model depth, and hidden layer dimensionality, with

all pipelines demonstrating functional encrypted inference feasibility despite varying performance trade-offs. All models are as accurate as their counterparts, due to the simple complexity of the training and data used, except for the encrypted border model, which could only sustain two convolutional layers due to TenSEAL CKKS constraints.

VI. Sustainability Analysis

A. Environmental

Deployment of FHE in ML workflows introduces increased computational overhead, particularly due to large ciphertext sizes, polynomial modulus degrees, and repeated encrypted multiplications. This translates to higher CPU and memory usage and longer inference times compared to plaintext models. From an environmental perspective, the additional compute demands could increase energy consumption and carbon footprint if scaled across large data centers. However, the current implementation only tested feasibility, meaning that models were only trained and tested on lightweight datasets which minimize impact. Furthermore, ability to perform secure inference on client devices without offloading sensitive data to cloud environments reduces the need for large scale centralized compute clusters, potentially offsetting environmental costs by lowering data movement and storage energy usage.

B. Social

The use of FHE in machine learning directly contributes to privacy preservation which is a critical component of social sustainability. Enabling inference on encrypted data provides a solution that protects sensitive data from exposure. This promotes trust in AI systems, encourages adoption in domains where privacy is critical and regulated, and aligns with principles of data sovereignty and human rights. The project and implemented pipelines demonstrate how advanced cryptographic methods can make ethical AI deployments feasible, especially in areas such as healthcare, law enforcement, and finance, where data misuse can have societal and economic consequences.

C. Economic

While encrypted inference incurs higher computational costs, it eliminates the need for trusted third parties to handle plaintext data. This can reduce compliance costs associated with data protection regulations (e.g., GDPR, HIPAA) and minimizes the risk of fines or reputational damage from breaches. Moreover, the solution highlights a pathway for secure machine learning services, which can expand into new markets where traditional AI is currently restricted due to privacy risks. The initial computational cost may therefore be balanced by long-term economic gains in terms of broader applicability and customer trust.

D. Applicability Justification

Although the environmental impact of FHE models is currently modest due to limited dataset sizes and small-scale experiments, scaling up would require explicit strategies such as energy-efficient homomorphic operations, optimized batching strategies, and hardware acceleration. The social and economic sustainability aspects are applicable, as privacy-preserving ML directly impacts trust, regulatory compliance, and adoption of secure AI solutions.

VII. Limitations and Considerations

Adapting a standard PyTorch CNN into a form compatible with the CKKS homomorphic encryption scheme required significant restructuring of the original model design. Non-linear functions such as ReLU, Sigmoid, or Tanh are not polynomial-friendly under FHE causing it to be replaced with polynomial approximation square activation. This limits the expressiveness of the model and can reduce accuracy compared to a plaintext equivalent. The border detection model was deliberately simplified to respect the multiplicative depth budget highlighting the trade-off between model complexity and feasibility under FHE constraints. An important design consideration is that the pipeline demonstrates a wrapper-based approach, meaning that PyTorch models could be adapted to work under FHE with sufficient restructuring. This provides a possibility for extending existing deep learning architectures to privacy-preserving inference without needing to redesign them entirely from scratch.

The project was constrained by several current limitations of TenSEAL. The first being that there is no bootstrapping support meaning computations were limited to a fixed modulus chain, requiring careful planning of multiplicative depth. Restricted ML layer compatibility also was a serious limitation with some standard layers such as pooling and non-polynomial activations not being directly supported leading to creating custom implementations. Another one was parameter tuning trade off with precision, performance, and security levels had to be balanced carefully by adjusting CKKS parameters. These limitations highlight that TenSEAL is not yet at a stage where large-scale deep learning models can be directly deployed under FHE without significant adaptations.

Another consideration and limitation that was identified and was the focus of the project, was that homomorphic encryption incurs heavy computational cost compared to plaintext inference. The performance overhead associated with FHE model inference as the encrypted convolutions were much slower than the plaintext counterparts due to the ciphertext expansion and noise growth that occurred. Per-sample encrypted inference times ranged from 1.2 seconds to 9 seconds. The ciphertexts are also substantially larger than the plaintext tensors, resulting in heavy memory usage. To keep inference feasible also, techniques such as input chunking into smaller encrypted vectors and reducing convolution depth were necessary to apply. Face Detection showed the highest per-sample cost despite using a comparable encryption scheme, poly modulus 8192, to MNIST, attributed to larger input resolution, 128x128 RGB and 28x28 grayscale, and 512-unit hidden layers. Memory consumption increased, with RAM usage ranging from 5.95 GB to 8.5, compared to typical plaintext CNN inference requiring under 2GB. For the border model, the difference between encrypted and validation MSE was

incredibly large, highlighting noise accumulation in regression tasks. To maintain feasibility, architectural simplifications were necessary. Input resolutions were reduced from 192x192→32x32 and 192x192→128x128 for border and face, respectively. Convolutional depth and layers were decreased due to identified limitation in TenSEAL's architecture. These constraints rendered real-time applications and deeper multilayer CNNs impractical under current CKKS implementations.

The current implementation employs a leveled FHE approach (fixed modulus chain), which results in computations being strained and inference capabilities being limited. Bootstrapping support would eliminate the need for architectural simplifications imposed by these depth constraints, allowing deployment of deeper convolutional networks with additional layers, high-level complex inputs, and more complex non-linear activation approximations. The theoretical benefits would include support for arbitrary circuit depth, residual neural networks, or transformer architectures without modification and removal of manual noise budget management, allowing automatic ciphertext refreshing as needed during inference. However, this would introduce substantial computational overhead. Further research into feasible bootstrapping support from libraries such as Concrete-ML, which provides capabilities for TFHE schemes, represents a promising direction for addressing leveled FHE limitations while balancing performance trade-offs inherent to ciphertext refreshing operations.

Despite these constraints and limitations, the project successfully achieved all six functional requirements (FR1-FR6) and all five non-functional requirements (NFR1-NFR5) (Table 1). FR1 was fulfilled through three trained ML models (MNIST, Face Detection, Border Generation) accepting encrypted inputs and producing encrypted predictions using TenSEAL's CKKS scheme. FR2 was demonstrated via encrypted arithmetic operations (addition, multiplication, convolution) implemented through TenSEAL library wrappers. FR3 was satisfied using PyTorch for model architecture development and TensorFlow for specific pipelines. FR4 was comprehensively addressed through extensive benchmarking comparing plaintext and encrypted models across execution time, accuracy, memory usage, and encryption overhead metrics. FR5 was realized through a dual-server web application enabling users to interact with encrypted inference pipelines via intuitive interfaces. FR6 was explored through optimizations including input dimensionality reduction, batching strategies, and circuit depth minimization. NFR1 was achieved via end-to-end CKKS lattice-based encryption of inputs and outputs with zero-trust dual-server architecture. NFR2 was addressed through privacy-preserving inference aligned with GDPR principles, eliminating plaintext data exposure. NFR3 was satisfied through clear web interface outputs displaying predictions, confidence scores, and visualization. NFR4 was fulfilled via modular pipeline architecture supporting extensibility to addition models and datasets. NFR5 was met through well-documented, modular codebase with configuration-driven design enabling future modifications and library upgrades. The complete fulfillment of all requirements demonstrates the technical feasibility of FHE-based privacy-preserving ML under current technological constraints.

Benchmarking was successfully conducted through direct comparison of plaintext and FHE model pipeline outputs and configurations. The project focused on demonstrating end-to-end feasibility, establishing plaintext-to-encrypted baselines within the same hardware and software environment and analysing trade-offs inherent to leveled CKKS implementations. This approach allowed for controlled evaluation of the computational overhead, accuracy degradation, and architectural constraints imposed by homomorphic encryption across discrete and continuous ML tasks using consistent infrastructure and parameter configurations. One limitation inherent to the project's benchmarking strategy, is comparison to other established FHE research performed using the CKKS scheme and CNN architectures. Contrasting results and architectures to previous work such as UniHENN [11] or CryptonDL [12] as a benchmark would allow further analysis of PrivML-FHE and how it fits into present work on FHE machine learning, instead of it serving as insight and lessons learned.

VIII. Conclusion and Future Work

This project demonstrates that privacy-preserving machine learning through Fully Homomorphic Encryption is feasible under current technological constraints, though significant practical challenges remain across classification and regression tasks. Three end-to-end encrypted inference pipelines, MNIST (97% accuracy, 0.9698 Precision Score, 1.601s application time), face detection (0.0036 MSE, 0.0444 MAE, 9.303s application time) and border generation (270.2084 MSE, 16.4367 MAE, 0.263s application time), validate that standard PyTorch models can be systematically adapted for FHE compatibility using CKKS encryption schemes. The completion of all functional and non-functional requirements confirms that leveled FHE approaches with carefully managed multiplicative depth budgets can enable meaningful encrypted computations without bootstrapping. However, evaluation reveals that fundamental trade-offs are inherent to current FHE-ML implementations such as computational overhead for larger noise budgets and architectural simplifications needed for CNN models.

Future directions should prioritize four key areas. The first is to integrate TCP/UDP based WebSocket which could enable secure, real-time-client-server communication of encrypted data streams. This would make it possible to demonstrate the model in a production format as an interactive privacy-preserving inference service, as well as provide a more conformed and appropriate web application. The second is looking into bootstrapping as a solution to the noise and limited multiplicative depth. Enabling support for bootstrapping and better tensor operations would allow deeper and more complex models to be supported. The current project shows how adapted PyTorch models can be adapted to FHE-compatible versions. Extending this into a general purpose "FHE wrapper toolkit" would allow developers to convert common Torch modulus into polynomial counterparts with minimal manual intervention. Combining federated learning with FHE could allow multiple users or devices to train and evaluate models collaboratively without exposing local data. This would be valuable for distributed sectors where privacy is critical.

Sustainability implications still must be considered when continuing this work. Environmentally, FHE's computational intensity raises energy consumption concerns for large-scale deployment, demanding hardware acceleration research, and algorithmic efficiency improvements. Socially, strong privacy guarantees address growing demands for ethical AI, regulatory compliance (GDPR, HIPAA), and user trust in data handling. Economically, while current costs limit adoption, privacy-preserving ML creates market opportunities in sectors where data confidentiality justifies computational overhead, particularly addressing enterprise concerns about AI service providers exploiting proprietary training data. Long term viability depends on bridging the performance gap through specialized hardware, optimized FHE schemes, and hybrid approaches balancing privacy guarantees with practical efficiency baselines and requirements.

ACKNOWLEDGEMENT

I would like to thank Arman Khouzani for his continuous help, support, insight, and feedback on this project.

REFERENCES

[1] "DeepSeek Cyber Attack Timeline," Cm-alliance.com, 2025. https://www.cm-alliance.com/deepseek-cyber-attack-timeline
[2] D. Bonderud, "Cost of a data breach in 2024 for the financial industry," IBM, Aug. 13, 2024. https://www.ibm.com/think/insights/cost-of-a-data-breach-2024-financial-industry
[3] M. Fazlioglu, "International Association of Privacy Professionals," iapp.org, Nov. 08, 2023. https://iapp.org/news/a/training-ai-on-personal-data-scraped-from-the-web
[4] R. Chatwood, "AI and intellectual property rights," Dentons.com, Jan. 28, 2025. https://www.dentons.com/en/insights/articles/2025/january/28/ai-and-intellectual-property-rights
[5] C. Gentry, "A Fully Homomorphic Encryption Scheme," 2009. https://crypto.stanford.edu/craig/craig-thesis.pdf
[6] R. Rivest, L. Adleman, and M. Dertouzos, "On Data Banks and Privacy Homomorphisms," 1978. https://luca-giuzzi.unibs.it/corsi/Support/papers-cryptography/RAD78.pdf
[7] S. Khot, "Hardness of approximating the shortest vector problem in lattices," Journal of the ACM, vol. 52, no. 5, pp. 789–808, Sep. 2005. https://doi.org/10.1145/1089023.1089027
[8] Z. Brakerski, C. Gentry, and V. Vaikuntanathan, "(Leveled) Fully Homomorphic Encryption without Bootstrapping," ACM TOCT, vol. 6, no. 3, pp. 1–36, Jul. 2014. https://doi.org/10.1145/2633600
[9] V. Lyubashevsky, C. Peikert, and O. Regev, "On Ideal Lattices and Learning with Errors over Rings," Journal of the ACM, vol. 60, no. 6, pp. 1–35, Nov. 2013. https://doi.org/10.1145/2535925
[10] J. Cheon, A. Kim, M. Kim, and Y. Song, "Homomorphic Encryption for Arithmetic of Approximate Numbers," 2016. https://eprint.iacr.org/2016/421.pdf
[11] "UniHENN: Designing More Versatile Homomorphic Encryption-based CNNs without im2col," Arxiv.org, 2024. https://arxiv.org/html/2402.03060v1
[12] A. Helbawy, M. Bahaa, and A. El Bolock, "CryptonDL: Encrypted Image Classification Using Deep Learning Models," 2023. https://www.scitepress.org/Papers/2023/120880/120880.pdf
[13] "Microsoft SEAL," GitHub, Nov. 27, 2022. https://github.com/Microsoft/SEAL
[14] S. Rogers, "PySEAL: Homomorphic encryption in a user-friendly Python package," Medium, May 23, 2018. https://medium.com/bioquest/pyseal-homomorphic-encryption-in-a-user-friendly-python-package-51dd6cb0411c
[15] A. Benaissá et al., "TenSEAL: A Library for Encrypted Tensor Operations Using Homomorphic Encryption," 2021. https://dp-ml.github.io/2021-workshop-ICLR/files/18.pdf
[16] "TenSEAL," GitHub, Jul. 24, 2023. https://github.com/OpenMined/TenSEAL
[17] M. Nocker et al., "HE-MAN - Homomorphically Encrypted Machine learning with oNnx models," Feb 16, 2023. https://arxiv.org/pdf/2302.08260
[18] "Introducing the Concrete Framework," Zama.ai, 2025. https://www.zama.ai/post/introducing-the-concrete-framework
[19] I. Goodfellow, Y. Bengio, and A. Courville, "Deep Learning," 2015. https://mcube.lab.nycu.edu.tw/~cfung/docs/books/goodfellow2016deep_learning.pdf
[20] PyTorch Contributors, "ReLU," Pytorch.org, 2023. https://docs.pytorch.org/docs/2.9/generated/torch.nn.ReLU.html
[21] "LeakyReLU — PyTorch 2.7 documentation," Pytorch.org, 2024. https://docs.pytorch.org/docs/stable/generated/torch.nn.LeakyReLU.html
[22] CodingNomads, "Mean Squared Error (MSE) Loss Function," Codingnomads.com, 2016. https://codingnomads.com/mean-squared-error-loss-function
[23] K. Pykes, "Cross-Entropy Loss Function in Machine Learning: Enhancing Model Accuracy," Datacamp.com, Jan. 11, 2024. https://www.datacamp.com/tutorial/the-cross-entropy-loss-function-in-machine-learnin
[24] M. Das, "Gradient Descent Variants: Explore Beyond Vanilla Gradient Descent," Medium, Aug. 24, 2023. https://medium.com/@HeCanThink/gradient-descent-variants-explore-beyond-vanilla-g
[25] P. Laubheimer, "Beyond the NPS: Measuring Perceived Usability with the SUS, NASA-TLX, and the Single Ease Question After Tasks and Usability Tests," Nielsen Norman Group, Feb. 11, 2018. https://www.nngroup.com/articles/measuring-perceived-usability/
[26] M. Soegaard, "System Usability Scale for Data-Driven UX," The Interaction Design Foundation, Nov. 17, 2023. https://www.interaction-design.org/literature/article/system-usability-scale?srsltid=AfmBOoqtjGy-H3t1OWNKn6H0dURHw3HPgrcEQ7W

All online resources were accessed on October 16, 2025.